Table of Contents

前言 1.1

第一章 DSO算法 1.2

- DSO算法理论介绍 1.2.1
- DSO算法框架 1.2.2
- 安装编译运行dso 1.2.3
- 安装编译运行dso_ros 1.2.4
- 在Xbot上运行DSO 1.2.5
- DSO安装编译运行常见错误及解决办法 1.2.6

第二章 SVO算法 1.3

- SVO算法理论介绍 1.3.1
- 安装编译运行SVO 1.3.2
- USB相机测试SVO 1.3.3
- 在Xbot上运行SVO 1.3.4

第三章 LSD算法 1.4

- LSD算法理论介绍 1.4.1
- 安装编译LSD 1.4.2
- 单目摄像头测试LSD 1.4.3
- 在Xbot上运行LSD 1.4.4

第四章 ORB-SLAM及ORB-SLAM2算法 1.5

- ORB-SLAM算法理论介绍 1.5.1
- 安装编译运行ORB-SLAM 1.5.2
- 安装编译运行ORB-SLAM2 1.5.3
- 实际摄像头测试ORB-SLAM及仿真实验 1.5.4
- ORB-SLAM代码介绍 1.5.5

第五章 VINS-Mono算法 1.6

- VINS-Mono算法理论介绍 1.6.1
- 安装编译运行VINS-Mono 1.6.2
- D435i运行VINS-Mono 1.6.3
- VINS-Mono安装编译运行常见错误及解决办法 1.6.4

VSLAM实践课程

本课程是对当前主流的VSLAM开源算法，比如DSO、LSD、ORB、SVO以及VIO算法VINS-Mono的实践操作课程，主要内容有：

- 各算法理论介绍
- 下载编译并利用公开数据集对算法进行验证
- 利用实际摄像头对算法进行验证
- 在xbot机器人上对算法进行验证
- 常见错误解决办法

DSO

本章首先来介绍DSO(Direct Sparse Odometry)算法，此算法是2016年由雅各布.恩格尔（Jakob Engel）博士发表，它属于稀疏直接法的视觉里程计。

本章首先对DSO算法进行了介绍，其次对代码框架也进行了简单的阐述，再讲解了如何安装编译此算法，并介绍了如何利用公开数据集以及实际摄像头来运行代码，以及展示了将算法部署到Xbot机器人上并进行了实地的测试效果，最后总结了一些上述过程中容易遇到的错误及解决办法。

2.1 DSO算法介绍

2.1.1 vSLAM现状

SLAM，即时定位与建图。广泛应用于机器人和无人驾驶等领域，从核心功能模块角度来讲，可以分为两类。基于激光雷达的激光SLAM和基于视觉的vSLAM。激光SLAM发展开始比较早，现在相关的理论和产品都已经比较成熟，例如扫地机器人、银行服务机器人等等。但是随着具有稀疏性的非线性优化理论(Bundle Adjustment)以及相机技术、计算性能的进步，vSLAM近年得到了快速的发展，实时运行vSLAM已经不再是梦想。

当前vSLAM算法有很多，比较主流的有稀疏法（ORB、PTAM、MonoSLAM），半稠密法（LSD-SALM、DSO、SVO），稠密法（DTAM、DVO），还有2018年最火爆的视觉与IMU融合的VINS等等。

2.1.2 特征点法和直接法

我们知道视觉SLAM主要分为前端视觉里程计（VO）和后端优化两部分。VO的实现方法，按照是否提取特征，分为特征点法和直接法。特征点法是视觉里程计的主流方法，它运行稳定，对光照，动态物体不敏感，算法也很成熟。而直接法则是不提取图像特征点，使用整体的像素信息进行位姿估计的vSLAM算法。比如特征点法的ORB，直接法的DSO。

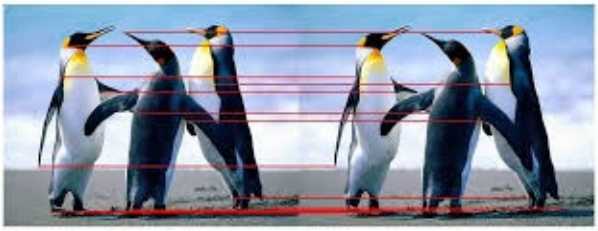
2.1.3 DSO是什么

DSO（Direct Sparse Odometry），是慕尼黑工业大学（Technical University of Munich, TUM）计算机视觉实验室的雅各布.恩格尔（Jakob Engel）博士，于2016年发布的一个视觉里程计方法。DSO属于稀疏直接法的视觉里程计。(目前单目开源)

DSO是少数使用纯直接法（Fully direct）计算视觉里程计的系统之一。

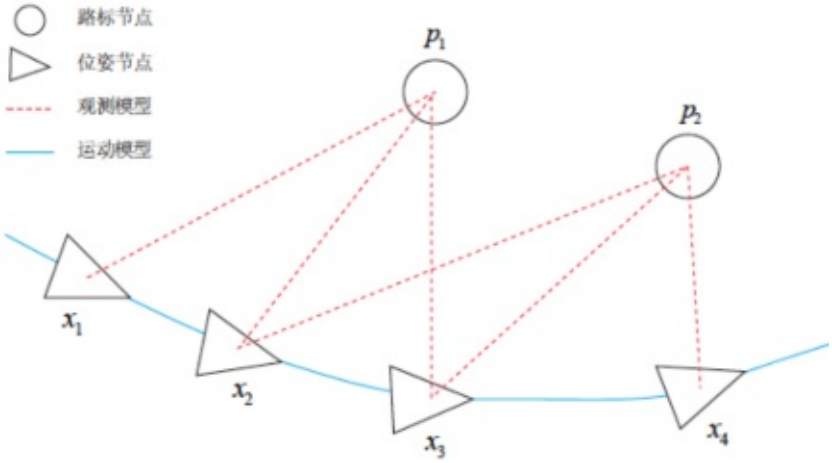
2.1.4 和特征点法的差异对比

直接法相比于特征点法，有两个非常不同的地方：特征点法通过最小化重投影误差来计算相机位姿与地图点的位置，而直接法则最小化光度误差（photometric error）



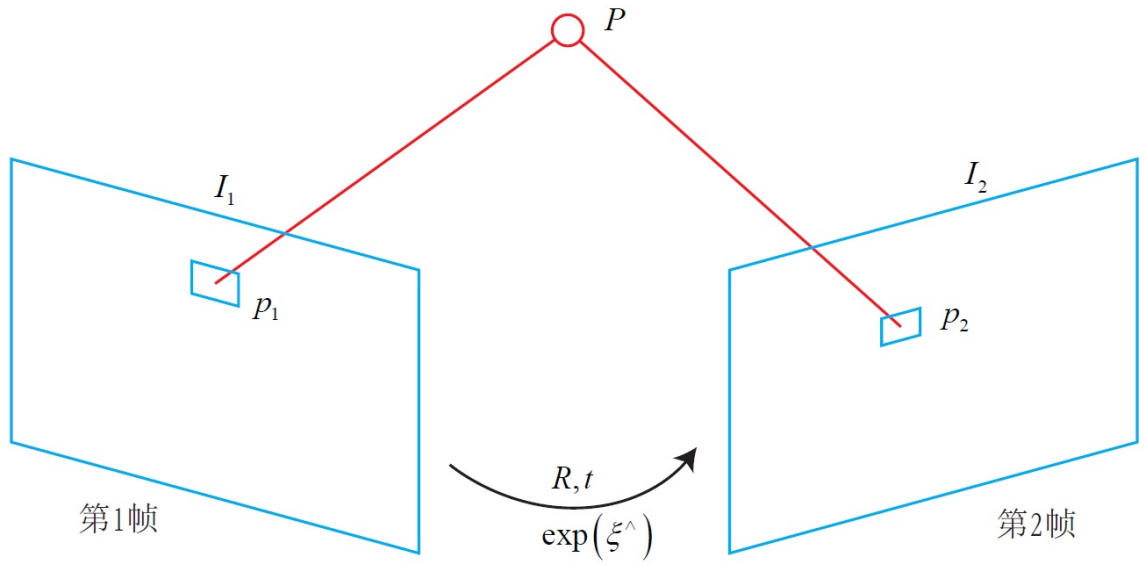
最小化重投影误差

经典的BA(Bundle Adjustment)问题，首先相机会进行一次投影，当前位姿估计出相机和路标点的坐标进行再次投影，两次投影之间的误差就是重投影误差，最小化重投影误差就是特征点法常用的后端位姿估计方法。



最小化光度误差

相对于最小化重投影误差，最小化目标函数是由图像之间的误差决定的，而不是重投影的误差，这种方法就是最小化光度误差。

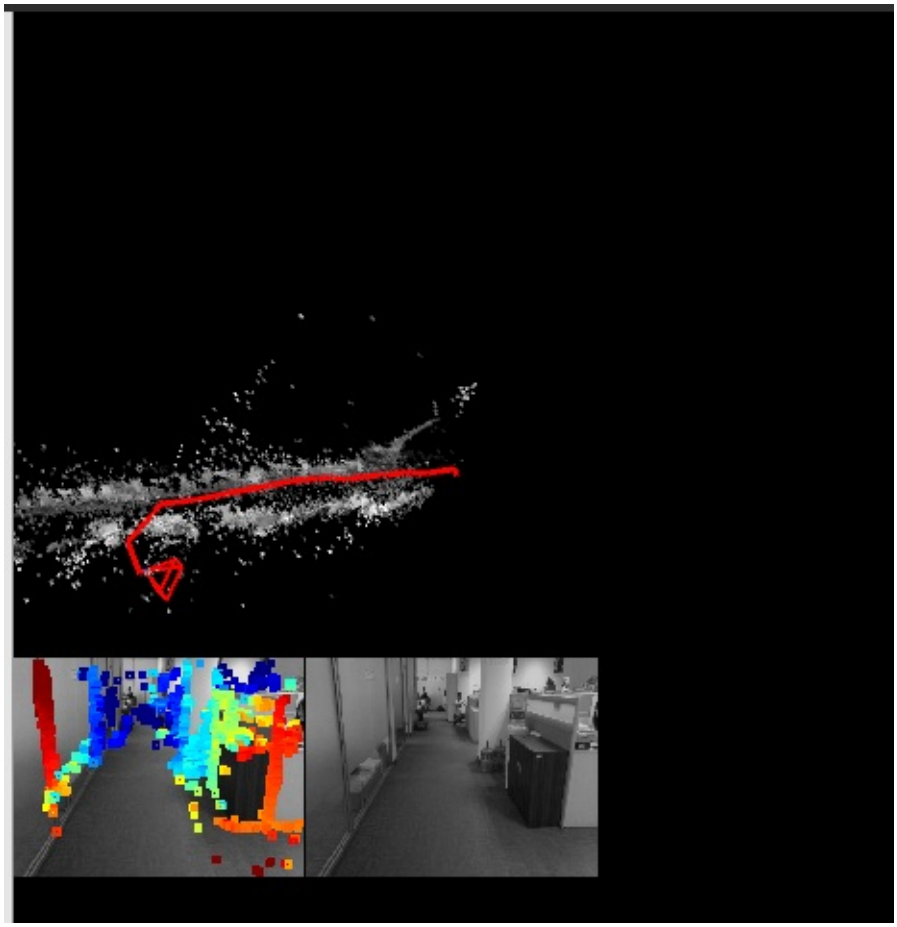


直接法将数据关联（data association）与位姿估计（pose estimation）放在了一个统一的非线性优化问题中，而特征点法则分步求解，即，先通过匹配特征点求出数据之间关联，再根据关联来估计位姿。这两步通常是独立的，在第二步中，可以通过重投影误差来判断数据关联中的外点，也可以用于修正匹配结果。

DSO我们很难将它划分为像特征点法那样一步一步的过程。DSO也没有“匹配点”这个概念。每一个三维点，从某个主导帧（host frame）出发，乘上深度值之后投影至另一个目标帧（target frame），从而建立一个投影残差（residual）。只要残差在合理范围内，就可以认为这些点是由同一个点投影的。DSO只要残差不大，就看成是同一个点。这是很重要的一点。在特征点法中，我们可以找到一个地图点分别在哪些帧中被看到，乃至找到各帧中的图像描述子是什么；但在DSO中，我们会尝试把每个点投影到所有帧中，计算它在各帧中的残差，而并不在意点和点之间的一一对应关系。

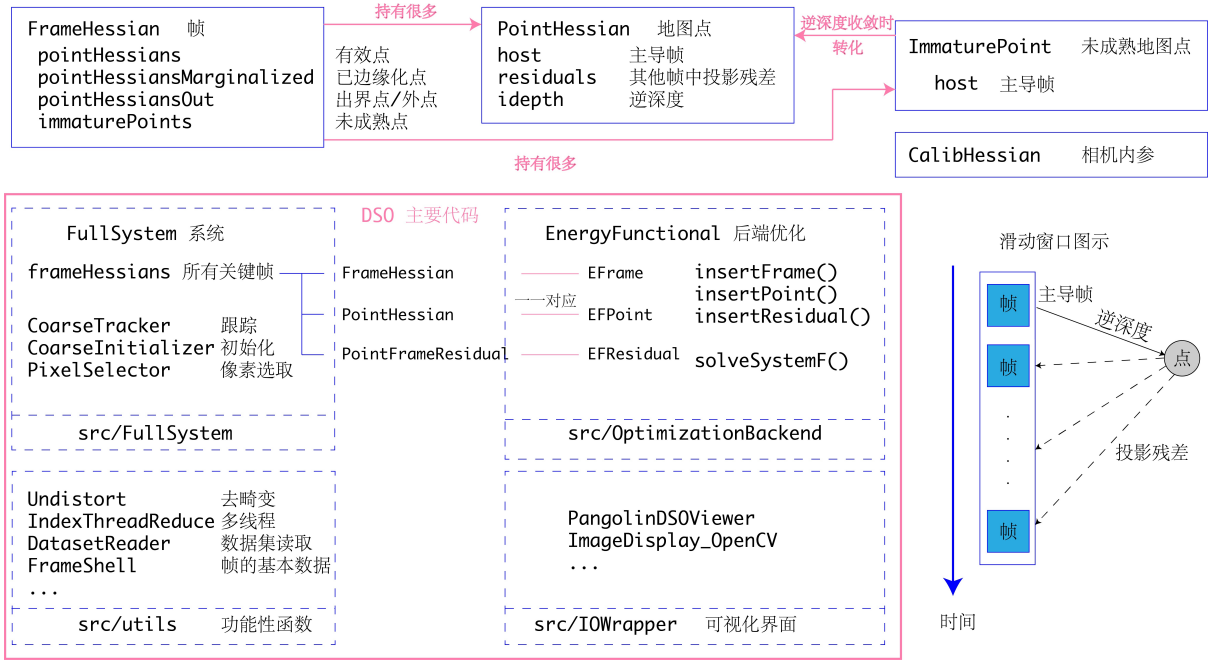
2.1.5 DSO的缺陷

由于直接法需要比较图像信息，其结果容易受光照干扰。于是，DSO提出了光度标定，认为对相机的曝光时间、暗角、伽马响应等参数进行标定后，能够让直接法更加鲁棒。对于未进行光度标定的相机，DSO也会在优化中动态估计光度参数。这个过程建模了相机的成像过程，因此对于由相机曝光不同引起的图像明暗变化，会有更好的表现。但是，如果由于环境光源发生变化，光度标定也是无能为力的。

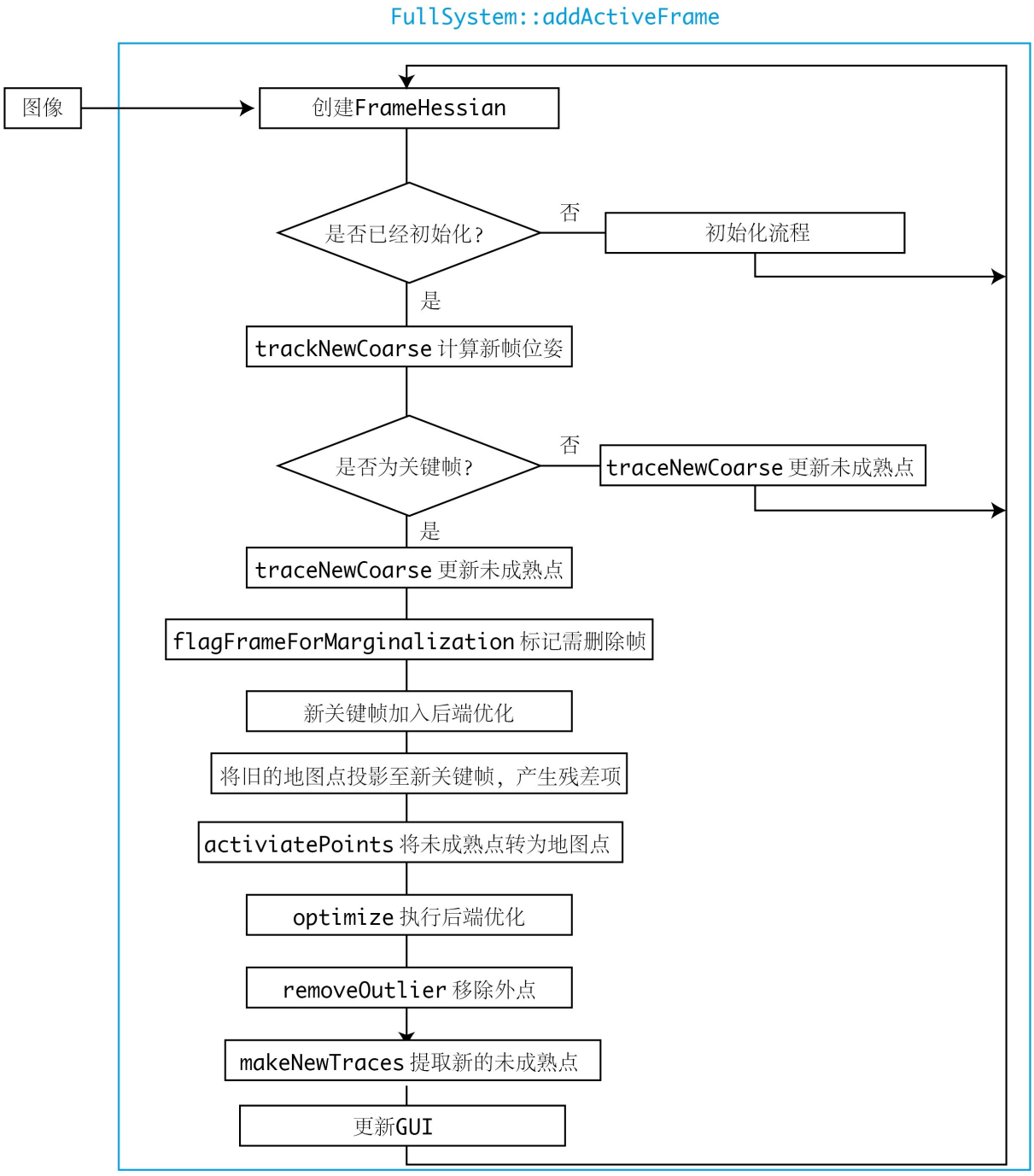


2.2 DSO算法框架

2.2.1 DSO代码框架



2.2.2 DSO算法流程



2.2.3 小结

- 对于非关键帧，DSO仅计算它的位姿，并用该帧图像更新每个未成熟点的深度估计；
- 后端仅处理关键帧部分的优化。除去一些内存维护操作，对每个关键帧主要做的处理有：增加新的残差项、去除错误的残差项、提取新未成熟点；
- 整个流程在一个线程内，但内部可能有多线程的操作。

参考文献

知乎：《DSO详解》--半闲居士(高翔)

2.3 DSO安装编译

2.3.1 编译环境

- Ubuntu 16.04
- ROS-kinetic

本课程以Ubuntu16.04为演示基础，课程代码也会兼容Ubuntu18.04。

2.3.2 安装流程全解

0. 依赖说明

DSO依赖项很少，只有Eigen3，pangolin，opencv等少数几个大家都已经很熟悉的库，作者的github页面上对安装DSO做了很详细说明，DSO下载以及安装链接https://github.com/JakobEngel/dso因此，安装DSO比较容易：

1. 安装eigen3

```
sudo apt-get install libsuitesparse-dev libeigen3-dev libboost-all-dev
```

2. 安装openCV.

DSO对opencv依赖很少，仅仅读或者写图像等一些简单操作。

```
sudo apt-get install libopencv-dev
```

3. 安装pangolin.

按照Pangolin的github主页https://github.com/stevenlovegrove/Pangolin

4. 安装ziplib.

```
sudo apt-get install zlib1g-dev
cd thirdparty #找到dso所在文件路径，切换到thirdparty文件夹下
tar -zxvf libzip-1.1.1.tar.gz
cd libzip-1.1.1/
./configure
make
sudo make install
sudo cp lib/zipconf.h /usr/local/include/zipconf.h
```

5. 安装编译DSO

首先可以单独穿件一个dso的工作空间：

```
mkdir -p ~/catkin_dso/src
cd catkin_dso/src
catkin_init_workspace
```

下载dso源码并编译：

```
git clone https://github.com/JakobEngel/dso.git
cd dso
mkdir build
cd build
cmake ..
make
```

2.3.3 脚本安装DSO

上述的安装方法比较全面的安装解析，如果对安装流程比较熟悉，不需要增加这方面的熟练度的化，可以使用本课程所提供的一键安装脚本。

1. 下载脚本

课程代码github地址TODO---

2. 执行命令

找到脚本所在位置，执行：

```
chmod + x installDSO.h
```
```
./installDSO.h
```

2.3.4 测试运行DSO

1.查看dso_dataset

```
cd /dso/build/bin
ls
```

如果存在 `dso_dataset` ,则说明DSO编译成功。

2.TUM单目数据集上测试运行DSO

TUM单目数据集官方链接：https://vision.in.tum.de/data/datasets/mono-dataset?redirect=1

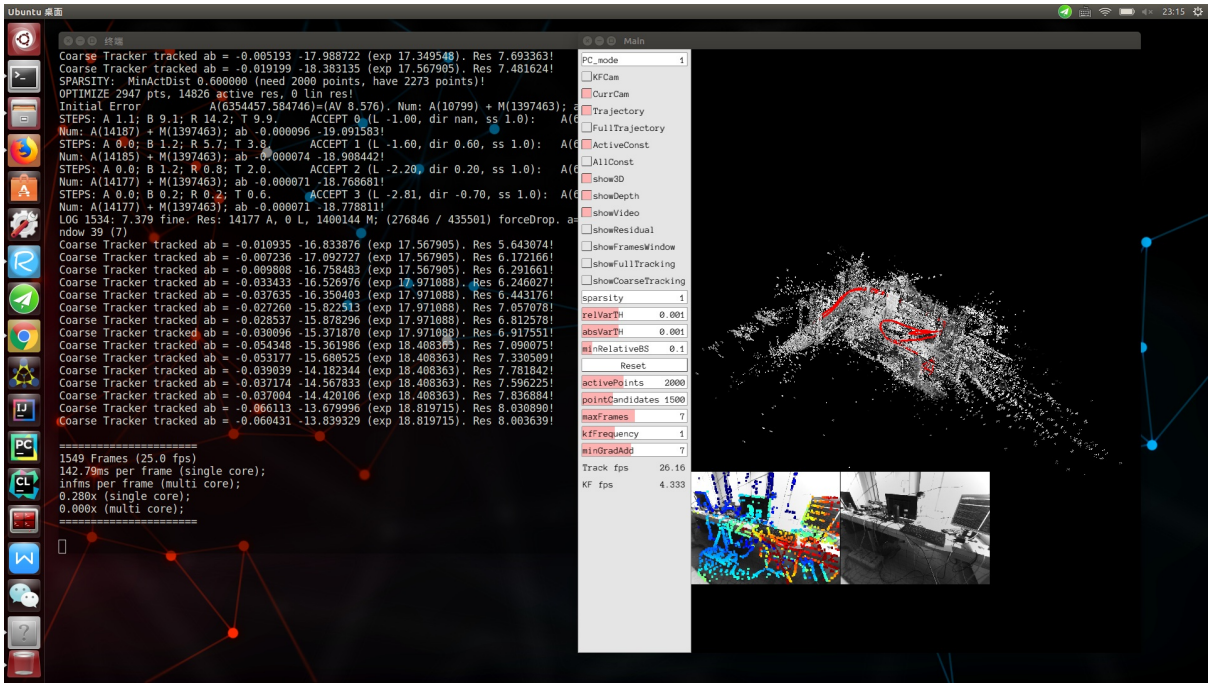
把下载的包解压到bin目录下,然后切换到DSO的bin目录下，使用下列命令执行程序。

```
cd /home/YOUR_PATH/dso/build/bin
./dso_dataset files=./sequence_12/images.zip calib=./sequence_12/camera.txt gamma=./sequence_12/pcalib.txt vign
ette=./sequence_12/vignette.png preset=0 mode=0
```

参数说明

- files为数据集图片压缩包；
- calib为相机内参数文件；
- gamma和vignette为相机的一些特性参数，光度标定文件；
- mode为DSO模式切换，如０为包含光度表达文件，１为只包含内参数，２为没有畸变参数；
- preset为设定DSO运行时的参数，如选取的像素点个数等等。preset＝3是preset＝０的5倍速运行DSO。

数据集运行结果如下：



至此，DSO在你的电脑上安装编译成功。

2.4 dso_ros的安装编译

2.4.1 编译环境

- Ubuntu 16.04
- ROS-kinetic
- 单目摄像头

本课程以Ubuntu16.04为演示基础，课程代码也会兼容Ubuntu18.04。

2.4.2 安装流程全解

1. 下载少Bug的dso_ros代码

```
cd ~/catkin_dso/src

git clone https://github.com/anchuanxu/dso_ros.git
export DSO_PATH=~/catkin_dso/src/dso
cd dso_ros
mkdir build
cd build
cmake ..
make
sudo make install
cd ~/catkin_dso
catkin_make
```

2. 安装usb_cam

```
echo "安装usb_cam......"
cd ~/catkin_dso/src
git clone https://github.com/bosch-ros-pkg/usb_cam.git
cd ..
catkin_make
source ~/catkin_dso/devel/setup.bash
```

2.4.3 脚本安装dso_ros

上述的安装方法比较全面的安装解析，如果对安装流程比较熟悉，不需要增加这方面的熟练度的化，可以使用本课程所提供的一键安装脚本。

1. 下载脚本

课程仓库github地址https://github.com/DroidAITech/vslam-evaluation.git

2. 执行命令

找到脚本所在位置，执行：

`chmod + x installdso_ros.h` `./installdso_ros.h`

2.4.4 相机标定

- ROSwiki有相机矫正的官方文档，有单目的也有立体相机的教程，官方链接:http://wiki.ros.org/camera_calibration
- 参考链接

1 下载标定板

标定板下载链接:http://wiki.ros.org/camera_calibration/Tutorials/MonocularCalibration?action=AttachFile&do=view&target=check-108.pdf

标定板为8x6,A4纸打印出来为，用直尺量square边长为24.5mm（这个根据自己标定板的大小自己量，大点好），即0.0245m，作为标定输入参数。

2 打开相机

```
cd ~/catkin_dso
source devel/setup.bash
roslaunch usb_cam usb_cam-test.get
# launch文件里面默认设备为/dev/video0,外接USB摄像头是有可能是/dev/video1，根据具体情况而定)
# 如果你没有矫正过的话，你会发现打开相机时会有一条警告
```

3 开始矫正

一般来说正常安装ros都是包含了camera_calibration，输入下面命令检查一下

```
~$ rosdep install camera_calibration
#All required rosdeps installed successfull
```

执行矫正文件

`rosrun camera_calibration cameracalibrator.py --size 8x6 --square 0.0245 image:=/usb_cam/image_raw camera:=/usb_cam`

弹出display窗口

注意：如果没有的话检查你命令输入是否输入正确，例如：8x6,中间不能用 "*" ,是字母 "x"，--size，--square前面是两个"-"，还要注意你的相机发出的话题是不是 `usb_cam/image_raw`

4 采集样本

- 使得标定板尽量出现在摄像头视野的各个位置里
- CALIBRATE按钮变绿色，点击等待
- 点击COMMIT将结果保存到电脑路径：/home/××××/.ros/camera_info/head_camera.yaml

5 结果格式例下

```
[image]
width
640
height
480
[narrow_stereo]
camera matrix
644.987121 0.000000 331.735116
0.000000 647.308571 248.505845
0.000000 0.000000 1.000000

distortion
0.248372 -0.436036 -0.008074 -0.000495 0.000000

rectification
1.000000 0.000000 0.000000
0.000000 1.000000 0.000000
0.000000 0.000000 1.000000

projection
669.478394 0.000000 331.064954 0.000000
```

```
0.000000 669.226440 245.232233 0.000000
0.000000 0.000000 1.000000 0.000000
```

参数意义：

- image_width、image_height代表图片的长宽 camera_name为摄像头名
- camera_matrix规定了摄像头的内部参数矩阵
- distortion_model指定了畸变模型
- distortion_coefficients指定畸变模型的系数
- rectification_matrix为矫正矩阵，一般为单位阵
- projection_matrix为外部世界坐标到像平面的投影矩阵

2.4.5 运行dso_ros

1.编写相机文件：

```
gedit ~/catkin_dso/src/dso_ros/camera.txt
```

格式如下：

```
 fx fy cx cy k1 k2 r1 r2
in_width in_height
"crop" / "full" / "fx fy cx cy 0"
out_width out_height
```

不同相机的格式写法不同，官方链接:https://github.com/JakobEngel/dso

2 执行命令

step1:打开一个终端

```
roscore
```

step2:再开终端

```
source ~/catkin_dso/devel/setup.bash
roslaunch usb_cam usb_cam-test.launch
```

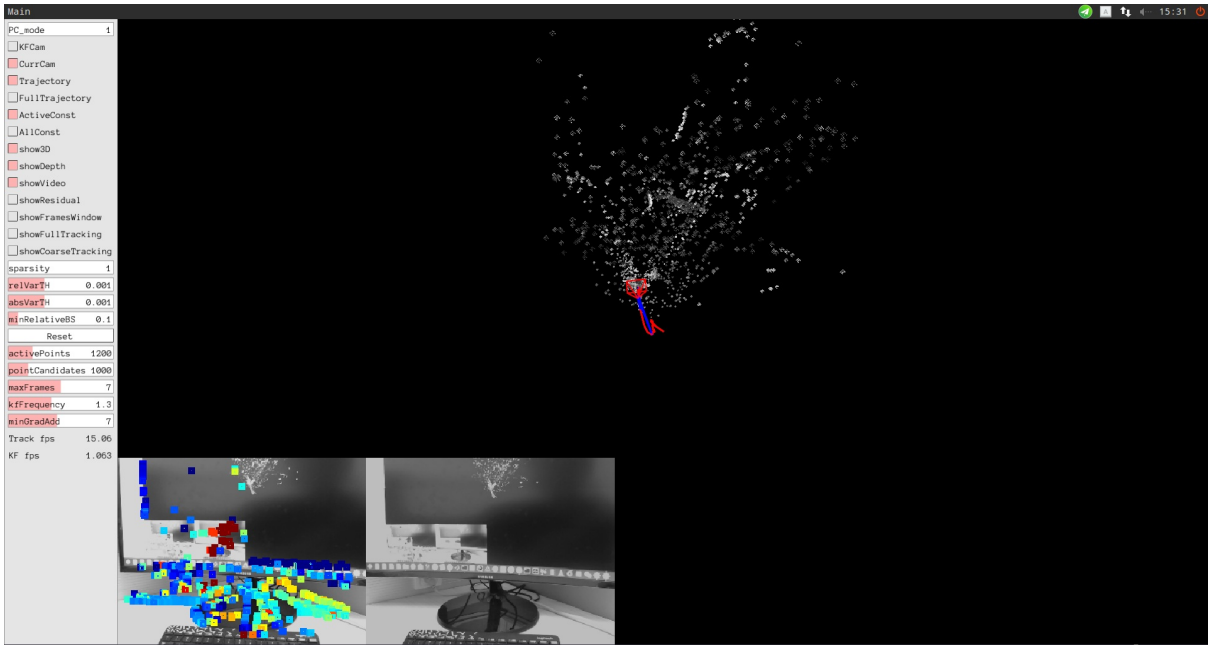
step3:再开终端

```
source ~/catkin_dso/devel/setup.bash
rosrun dso_ros dso_live image:=/usb_cam/image_raw calib=/home/YOUR_PATH/camera.txt mode=1
```

2.4.6 跑通图示

单目摄像头初始化截图：

备注：摄像头分辨率：`640 X 480`



至此，dso_ros安装编译也顺利完成，可以愉快的用自己的摄像头玩耍DSO了。

2.5 DSO在仿真机器人的运行

2.5.1 仿真环境和仿真机器人

课程的仿真环境，我们利用中国大学Mooc《机器人操作系统入门》课程代码的仿真环境，顺便一提，该课程作为我司优秀的ROS入门课程，课程质量比较优秀，非常适合ROS入门学习。近期，课程代码包进行了全面的更新维护，新支持了Ubuntu18.04，fixed了一些bug，有兴趣可以去探索一下，这里不再赘述。

1. 下载编译ROS-Academy-for-Beginners代码包

- 新建工作空间

```
mkdir -p ~/catkin_rafb/src
cd ~/catkin_rafb/src
catkin_init_workspace
cd ..
catkin_make
```

- 下载代码包 ``` cd ~/catkin_rafb/src git clone https://github.com/DroidAITech/ROS-Academy-for-Beginners

如果是Ubuntu18.04,则将仓库分支切换成melodic

```
- 安装教学包所需的依赖
```

$ cd ~/catkin_ws $ rosdep install --from-paths src --ignore-src --rosdistro=kinetic -y

```
- 编译并刷新环境
```

$ catkin_make $ source ~/catkin_ws/devel/setup.bash

```
**注意：** 按照README.md提示升级gazebo，否则将会出现运行卡顿问题。

### 2.5.2 修改usb_cam的接入话题

由于在仿真环境中，xbot所携带摄像头发布话题并非`usb_cam`，所以要更改相关配置：

打开仿真环境:
```

cd catkin_rafb/ # 包含仿真环境的工作空间 source devel/setup.bash roslaunch robot_sim_demo robot_spawn.launch

```
重新打开一个终端,查看当前工作相机发布话题
```

rostopic list

/camera_D415/rgb/image_raw 是我们要的发布图像信息话题,或者为

/camera/rgb/image_raw ， 所下载仓库版本不同，话题可能稍有不同。

```
再打开一个终端
```

cd catkin_dso/src/usb_cam/launch gedit usb_cam-test.launch

修改launch文件如下:

```
# 图片长和宽修改为xbot相机参数值
<param name="image_width" value="1280" />
<param name="image_height" value="720" />
<param name="pixel_format" value="yuyv" />
<param name="camera_frame_id" value="usb_cam" />
<param name="io_method" value="mmap"/>
```

</node>

重映射xbot现在发布的话题

```
<remap from="image" to="/camera_D415/rgb/image_raw"/>
<param name="autosize" value="true" />
```

</node> </launch>

```
保存后测试usb_cam能够接收到xbot上摄像头发布的Image.
```

cd ~/catkin_dso source devel/setup.bash roslaunch usb_cam usb_cam-test.launch

```
新建xbot仿真摄像头的camera.txt如下:
```

cd ~ gedit camera.txt

写入：

1108.7654256452881 1108.7654256452881 640.5 360.5 0 0 0 0 1280 720 crop 640 480

```
保存完成，关闭所有终端。

### 2.5.3 运行仿真机器人，用DSO建图

##### 1. 启动仿真环境
```

cd ~/catkin_rafb source devel/setup.bash roslaunch robot_sim_demo robot_spawn.launch

```
##### 2. 启动usb_cam
```

cd ~/catkin_dso source devel/setup.bash roslaunch usb_cam usb_cam-test.launch

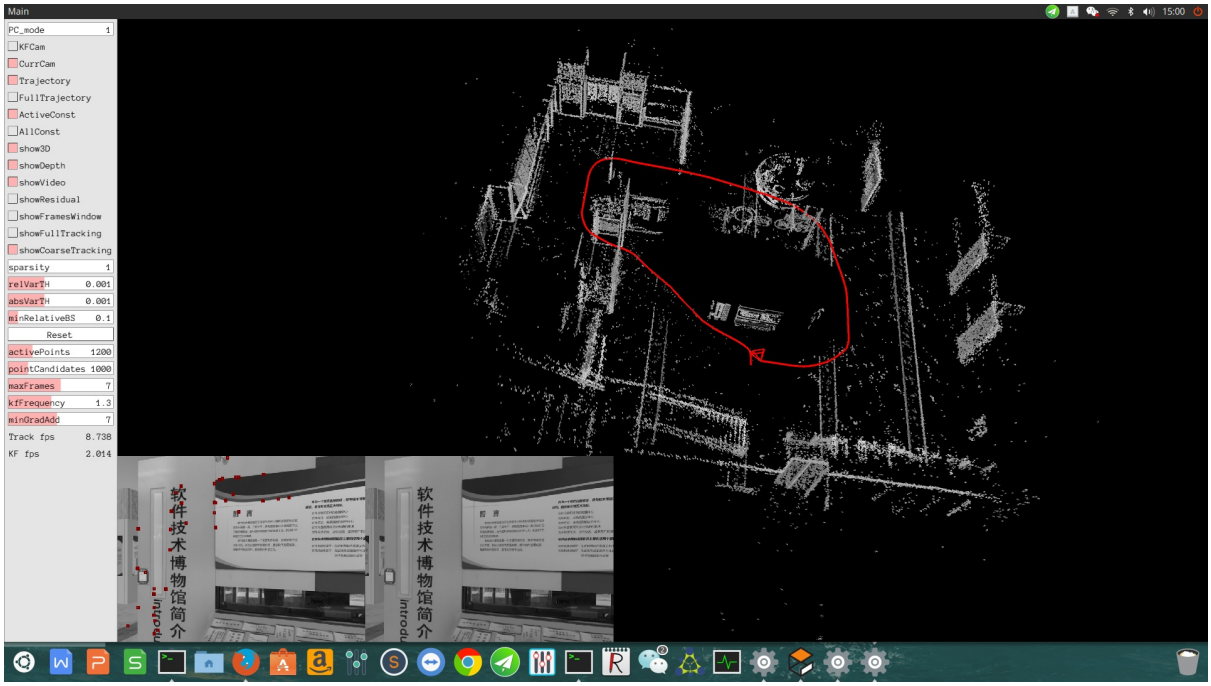
```
##### 3. 启动dso_ros

再开一个终端
```

cd catkin_dso source devel/setup.bash rosrun dso_ros dso_live image:=/camera_D415/rgb/image_raw calib=/home/YOUR_PATH/camera.txt

```
##### 4. 启动机器人键盘控制
再开一个终端
```

cd catkin_ws/ #仿真环境所在工作空间 source devel/setup.bash rosrun robot_sim_demo robot_keyboard_teleop.py #启动键盘控制程序 ``` 移动机器人进行DSO建图。

2.5.4 仿真建图效果



2.6 DSO安装编译运行常见错误和解决办法

2.6.1 DSO安装与编译

遇到的问题与解决

`error 1:`

运行 `./dso_dataset fileXXXXX` 时候出现

```
=============== PRESET Settings: ===============
DEFAULT settings:
- no  real-time enforcing
- 2000 active points
- 5-7 active frames
- 1-6 LM iteration each KF
- original image resolution
==============================================
PHOTOMETRIC MODE WITHOUT CALIBRATION!
ERROR 9 reading archive /home/YOUR_PATH/dso/built/bin/sequence_12/images.zip!
```

error1解决办法

因为 `sudo apt-get install zlib1g-dev` 安装的解压版本没更新,可能已经更新了还不行,则手动解压TUM数据序列集合中的 `images.zip` ,然后在 `./dso_dataset fileXXXXX` 中也修改为:

```
./dso_dataset files=./sequence_12/images calib=./sequence_12/camera.txt gamma=./sequence_12/pcalib.txt vignette
=./sequence_12/vignette.png preset=0 mode=0
```

即可.

`error 2:`

运行 `./dso_dataset fileXXXXX` 时候出现

```
Reading Calibration from file  ... not found. Cannot operate without calibration, shutting down.
段错误 (核心已转储)
```

error2解决办法

因为 指令中的路径写错,检查路径后修正即可跑通.

2.6.2 dso_ros安装编译

可能遇到的问题

`error1`

```
/usr/bin/ld: CMakeFiles/dso_live.dir/src/main.cpp.o: undefined reference to symbol '_ZN5boost6system15system_ca
tegoryEv'
/usr/lib/x86_64-linux-gnu/libboost_system.so: error adding symbols: DSO missing from command line
collect2: error: ld returned 1 exit status
CMakeFiles/dso_live.dir/build.make:198: recipe for target 'bin/dso_live' failed
make[3]: *** [bin/dso_live] Error 1
```

error1解决办法

```
cd ~/catkin_dso/src/dso_ros
gedit CMakeLists.txt
# 编辑前：
target_link_libraries(dso_live ${DSO_LIBRARY} ${Pangolin_LIBRARIES} ${OpenCV_LIBS})

# 编辑后：
target_link_libraries(dso_live ${DSO_LIBRARY} ${Pangolin_LIBRARIES} ${OpenCV_LIBS} GL glut GLU boost_system)
```

`error2`

```
# 运行dso_ros出现

Couldn't find executable named dso_live below /home/chuanxu/catkin_dos/src/dso_ros
```

error2解决办法

- 可能原因1：下错了版本,下成了rosbuild,重新下载catkin版本并编译测试。
- 可能原因2：rosrun ××× 路径错误，找不到camera.txt，检查路径并修正即可。

`error3`

xbot上安装 usb_cam 出现error:

```
[usb_cam-1] process has died [pid 5257, exit code 1, cmd /home/zkzdzn/catkin_dso/devel/lib/usb_cam/usb_cam_node
 __name:=usb_cam __log:=/home/zkzdzn/.ros/log/ea6db8de-6759-11e9-a323-34415de66a2c/usb_cam-1.log].
log file: /home/zkzdzn/.ros/log/ea6db8de-6759-11e9-a323-34415de66a2c/usb_cam-1*.log
```

error3解决办法

- `rostopic list`
- 找到自己相机发布的image topic 例如： /camera/color/image_raw
- 找到usb_cam launch文件 修改重映射 remap 到自己相机发布的topic

```
</node>
  <node name="image_view" pkg="image_view" type="image_view" respawn="false" output="screen">
    # 修改前
    <remap from="image" to="/usb_cam/image_raw"/>
    # 修改后
    <remap from="image" to="/camera/color/image_raw"/>
    <param name="autosize" value="true" />
  </node>
</launch>
```

2.6.3 其他

有更多相关问题欢迎到课程官方github开issue交流讨论

- github地址：https://github.com/DroidAITech/vslam-evaluation.git

SVO

本章我们介绍的是SVO(Semi-Direct Monocular Visual Odometry)算法，是苏黎世大学Scaramuzza教授的实验室在2014年发表的一种视觉里程计算法，一般称为半直接法，也称稀疏直接法。

本章先对SVO理论部分有一个简单的介绍，再分别介绍利用公开数据集以及笔记本摄像头对SVO算法进行测试的流程，最后将SVO算法实际部署到Xbot机器人上并进行了实地的实验。

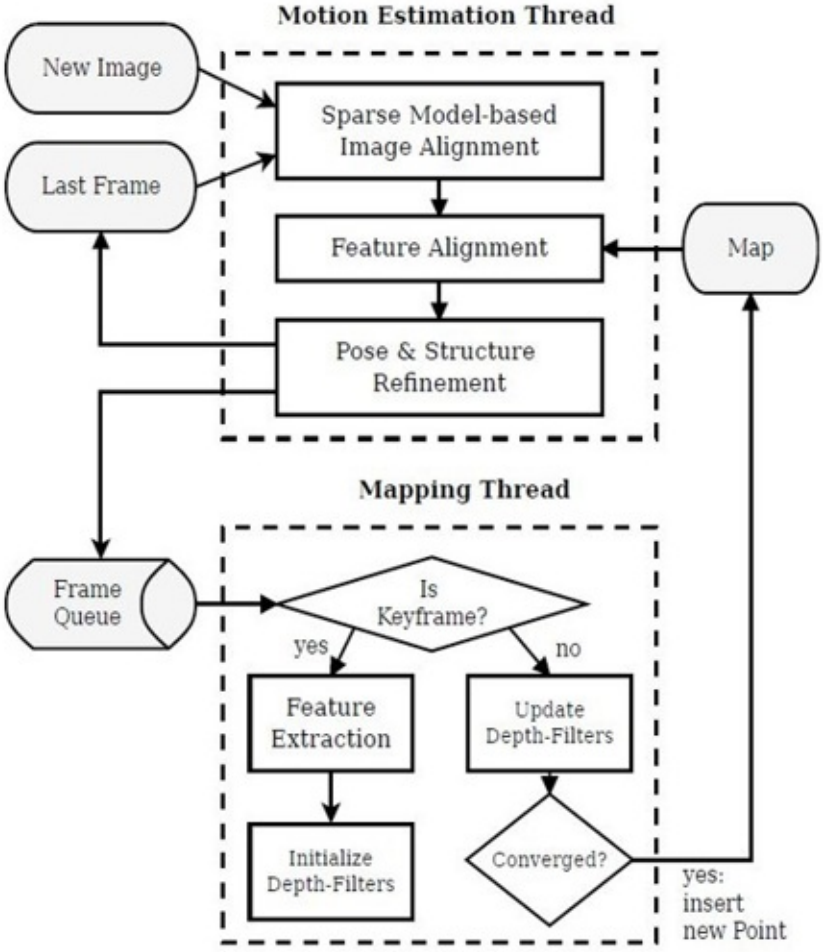
2.1 理论知识

2.1.1 SVO简介

SVO（Semi-Direct Monocular Visual Odometry）半直接法视觉里程计，是苏黎世大学机器人感知组的克里斯蒂安弗斯特等人2014年在ICRA会议上发表的工作。所谓半直接法就是混合使用直接法与特征点法，SVO首先像特征法那样提取了一些关键点（角点，没有描述子），然后像直接法那样，根据关键点周围的信息估计相机的运动及位置。

- 算法特点
  1.SVO作为一个VO（Visual Odometry）算法省去了回环检测和重定位的功能，不需要特征点描述子的计算描述子，因此算法速度极快，在PC机上的图像处理能达到300 frames per second（fps），即使在低端计算平台上也能做到实时性。
  2.SVO是一个VO算法，不是SLAM，没有闭环检测。

SVO算法主要分为Tracking与Mapping两个部分,其中
1.Tracking部分主要是估计当前帧的位姿,
2.Mapping部分主要负责计算特征点的深度。

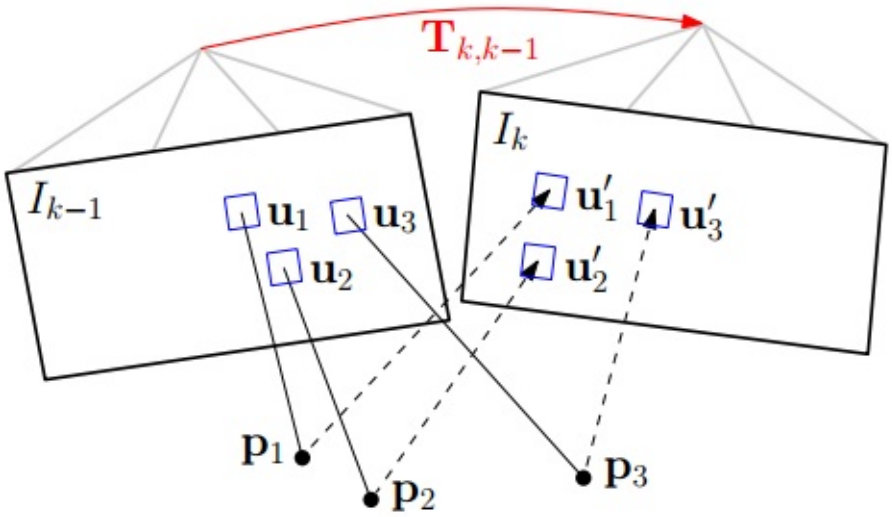


2.1.2 Tracking

Tracking部分主要任务是估计当前帧的位姿，主要干了两件事：

- 1.通过图像对齐_计算当前帧的（粗略）位姿
  ①.找到前帧（K-1）看到的地图点p1,p2,p3；

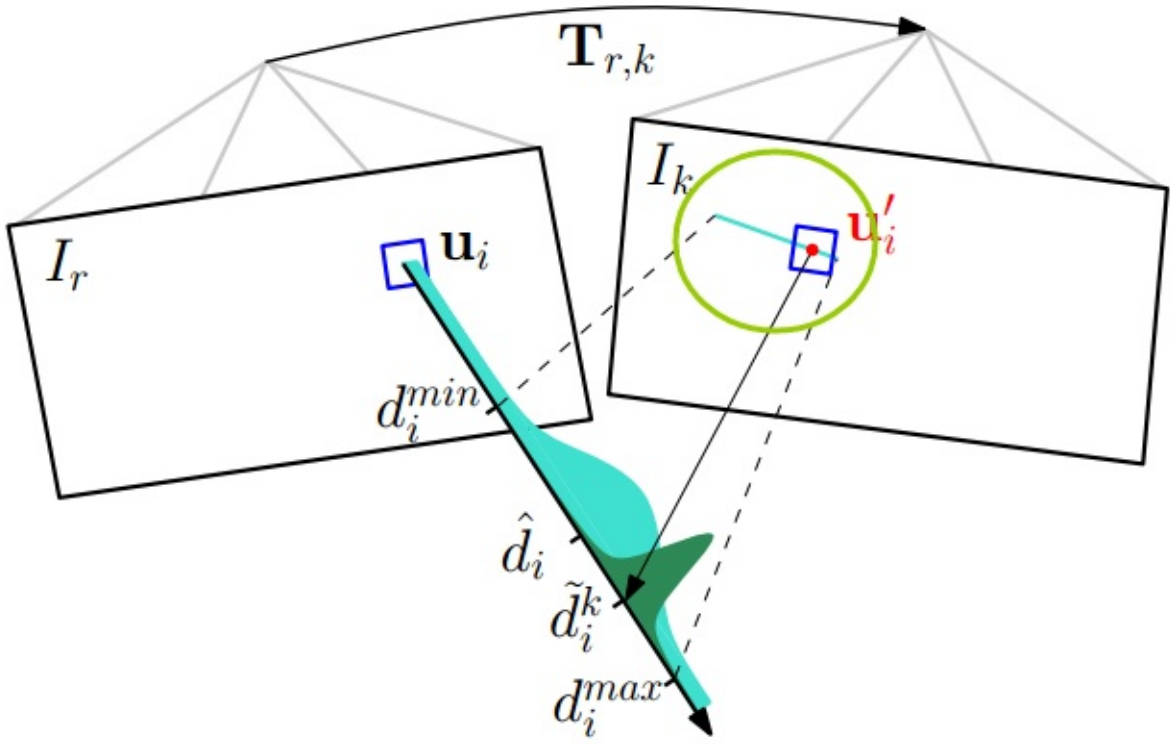
②.投影至后帧（k）二维图像上；
③.然后最小化灰度误差函数得到位姿；



- 2.根据地图，对位姿进一步优化:
  当前帧位姿的估计是基于上一帧的结果来计算的，所以如果把它当作真实位姿估计的话，将有较大的累积误差。因此，需要进一步和地图之间进行特征点比对，来对当前帧位姿进一步优化。

2.1.3 Mapping

单目VO中，刚刚从图像中提取的关键点是没有深度信息，利用一系列的前后帧，通过三角化估计深度完成对指定帧上特征点深度的求取。



SVO采用主要是极线搜索确定匹配点，通过参考帧中特征点图像位置的深度值计算出他在当前帧中的极线段位置，进行特征搜索匹配。通过确定特征点位置来三角化计算深度。
工作流程：关键帧上选取特征点(SVO是fast角点)，作为种子（就是深度未收敛的特征点），每来一帧图像，更新一下之前提取种子的深度信息，直至种子的深度收敛，并把它放到地图中，供Tracking线程使用。如果新的一帧是关键帧，那么再次提取新的特征点作为种子。就这样旧的种子不断收敛，新的种子不断增加，不断进行下去。

2.2 SVO实际部署

2.2.1 运行环境

部署环境介绍

- Ubuntu 16.04
- ROS-kinetic 安装教程
- Xbot仿真平台 部署教程

2.2.2 安装部署

- **创建SVO工作空间**

```
$ mkdir -p ~/SVO_demo/src #SVO_demo为SVO工作空间
$ cd ~/SVO_demo/src
$ catkin_init_workspace
```

- **安装Sophus**

```
$ git clone https://github.com/strasdat/Sophus.git
$ cd Sophus
$ git checkout a621ff
$ mkdir build
$ cd build
$ cmake ..
$ make
```

- **安装Fast**

```
$ cd ~/SVO_demo/src
$ git clone https://github.com/uzh-rpg/fast.git
$ cd fast
$ mkdir build
$ cd build
$ cmake ..
$ make
```

- **安装g2o**

```
$ cd ~/SVO_demo/src
$ git clone https://github.com/RainerKuemmerle/g2o.git
$ cd g2o
$ mkdir build
$ cd build
$ cmake ..
make
$ sudo make install
```

- **安装vikit**

```
$ cd ~/SVO_demo/src
$ git clone https://github.com/uzh-rpg/rpg_vikit.git
```

- **安装依赖**

```
$ sudo apt-get install ros-kinetic-cmake-modules
```

- **下载SVO源码**

```
$ cd ~/SVO_demo/src
$ git clone https://github.com/uzh-rpg/rpg_svo.git
$ sed -i 's/from python_qt_binding.QtGui import QWidget/from python_qt_binding.QtWidgets import QWidget/' ~/SVO
_demo/src/rpg_svo/rqt_svo/src/rqt_svo/svo_widget.py #修改一个bug
```

- **编译**

```
$ cd  ~/SVO_demo/
$ catkin_make
```

2.2.3 运行SVO及数据集

下载数据集

```
$ cd ~/SVO_demo/src
$ wget rpg.ifi.uzh.ch/datasets/airground_rig_s3_2013-03-18_21-38-48.bag
```

启动SVO

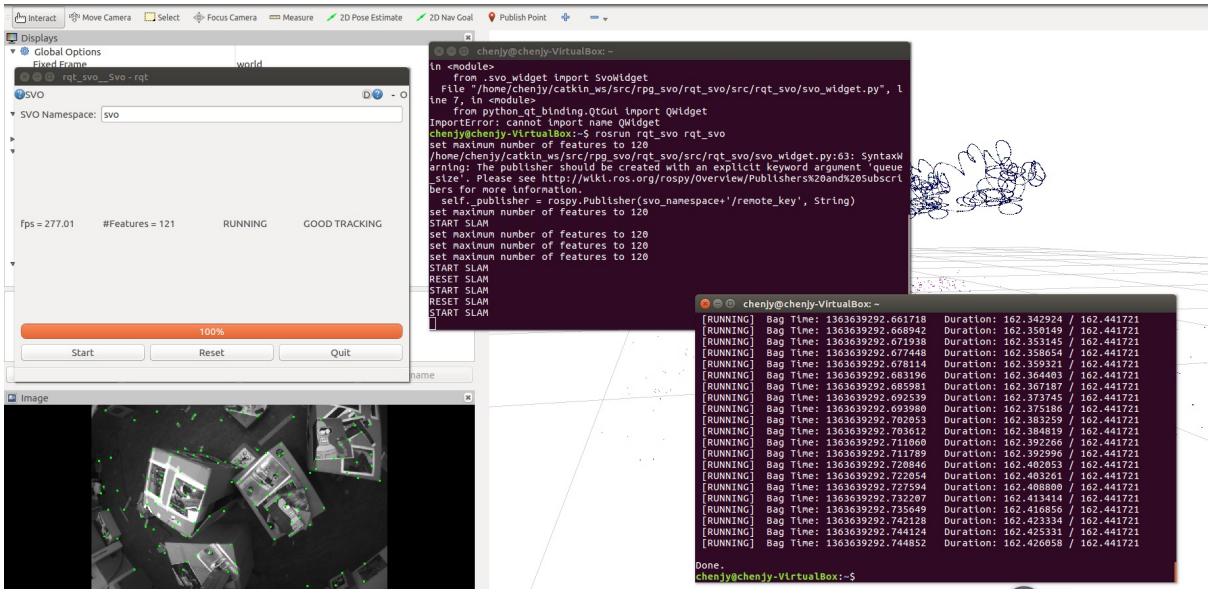
```
# 打开新终端
$ cd  ~/SVO_demo/  #SVO_demo为SVO的工作空间
$ source devel/setup.bash  #刷新环境
$ roslaunch svo_ros test_rig3.launch
# 启动rviz：打开新终端
$ cd  ~/SVO_demo/  #SVO_demo为SVO的工作空间
$ source devel/setup.bash  #刷新环境
$ rosrun rviz rviz -d ~/SVO_demo/src/rpg_svo/svo_ros/rviz_config.rviz
# 启动GUI界面：再打开新终端
$ cd  ~/SVO_demo/  #SVO_demo为SVO的工作空间
$ source devel/setup.bash  #刷新环境
$ rosrun rqt_svo rqt_svo
# SVO NameSpace栏输入svo
```

播放rosbag

下载的数据集所在文件夹打开终端：

$ rosbag play airground_rig_s3_2013-03-18_21-38-48.bag

运行截图如下：


Interact
Move Camera
Select
Focus Camera
Measure
2D Pose Estimate
2D Nav Goal
Publish Point
Displays
Global Options
Fixed Frame
world
rqt_svo__Svo - rqt
SVO
SVO Namespace:
svo
fps = 277.01
#Features = 121
RUNNING
GOOD TRACKING
100%
Start
Reset
Quit
Image
chenjy@chenjy-VirtualBox: ~
in <module>
from .svo_widget import SvoWidget
File "/home/chenjy/catkin_ws/src/rpg_svo/rqt_svo/src/rqt_svo/svo_widget.py", l
ine 7, in <module>
from python_qt_binding.QtGui import QWidget
ImportError: cannot import name QWidget
chenjy@chenjy-VirtualBox:~$ rosrun rqt_svo rqt_svo
set maximum number of features to 120
/home/chenjy/catkin_ws/src/rpg_svo/rqt_svo/src/rqt_svo/svo_widget.py:63: SyntaxW
arning: The publisher should be created with an explicit keyword argument 'queue
_size'. Please see http://wiki.ros.org/rospy/Overview/Publishers%20and%20Subscri
bers for more information.
self._publisher = rospy.Publisher(svo_namespace+'/remote_key', String)
set maximum number of features to 120
START SLAM
set maximum number of features to 120
set maximum number of features to 120
set maximum number of features to 120
START SLAM
RESET SLAM
START SLAM
RESET SLAM
START SLAM
chenjy@chenjy-VirtualBox: ~
[RUNNING] Bag Time: 1363639292.661718 Duration: 162.342924 / 162.441721
[RUNNING] Bag Time: 1363639292.668942 Duration: 162.350149 / 162.441721
[RUNNING] Bag Time: 1363639292.671938 Duration: 162.353145 / 162.441721
[RUNNING] Bag Time: 1363639292.677448 Duration: 162.358654 / 162.441721
[RUNNING] Bag Time: 1363639292.678114 Duration: 162.359321 / 162.441721
[RUNNING] Bag Time: 1363639292.683196 Duration: 162.364403 / 162.441721
[RUNNING] Bag Time: 1363639292.685981 Duration: 162.367187 / 162.441721
[RUNNING] Bag Time: 1363639292.692539 Duration: 162.373745 / 162.441721
[RUNNING] Bag Time: 1363639292.693980 Duration: 162.375186 / 162.441721
[RUNNING] Bag Time: 1363639292.702853 Duration: 162.383259 / 162.441721
[RUNNING] Bag Time: 1363639292.703612 Duration: 162.384819 / 162.441721
[RUNNING] Bag Time: 1363639292.711060 Duration: 162.392266 / 162.441721
[RUNNING] Bag Time: 1363639292.711789 Duration: 162.392996 / 162.441721
[RUNNING] Bag Time: 1363639292.720846 Duration: 162.402053 / 162.441721
[RUNNING] Bag Time: 1363639292.722054 Duration: 162.403261 / 162.441721
[RUNNING] Bag Time: 1363639292.727594 Duration: 162.408800 / 162.441721
[RUNNING] Bag Time: 1363639292.732207 Duration: 162.413414 / 162.441721
[RUNNING] Bag Time: 1363639292.735649 Duration: 162.416856 / 162.441721
[RUNNING] Bag Time: 1363639292.742128 Duration: 162.423334 / 162.441721
[RUNNING] Bag Time: 1363639292.744124 Duration: 162.425331 / 162.441721
[RUNNING] Bag Time: 1363639292.744852 Duration: 162.426058 / 162.441721
Done.
chenjy@chenjy-VirtualBox:~$

2.3 SVO实际部署--运行SVO+USB_camera

2.3.1 安装摄像头驱动usb_cam

```
# 打开新终端
$ mkdir -p ~/catkin_USB_cam/src  #创建USB_cam工作空间catkin_USB_cam
$ cd  ~/catkin_USB_cam/src
$ git clone https://github.com/ros-drivers/usb_cam.git  #下载源码
$ cd ..
$ catkin_make  #编译
```

2.3.2 相机标定

- 打印一张标定板
- 打开相机

  ```
  # 打开新终端
  $ cd ~/catkin_USB_cam  #Scatkin_USB_cam为USB_camera的工作空间
  $ source devel/setup.bash  #刷新环境
  $ roslaunch usb_cam usb_cam-test.launch
  ```

- 打开矫正窗口

  ```
  $ rosrun camera_calibration cameracalibrator.py --size 8x6 --square 0.0245 image:=/usb_cam/image_raw camera
  :=/usb_cam
  ```

- 移动标定板
  不断移动标定板，直到CALIBRATE按钮变亮
- 计算矫正参数
  点击CALIBRATE按钮就会进行标定（标定过程将持续一两分钟）。并可以在命令窗中看到标定参数，点击COMMIT将结果保存到电脑。

2.3.3 修改配置参数

修改rpg_svo/svo_ros//param/camera_atan.yaml文件参数为标定得到的对应参数
修改rpg_svo/svo_ros/launch/live.launch文件参数（将订阅的topic改为USB_camer发布的topic)将第六行改为

2.3.4 启动SVO+USB_camera

启动USB_camera

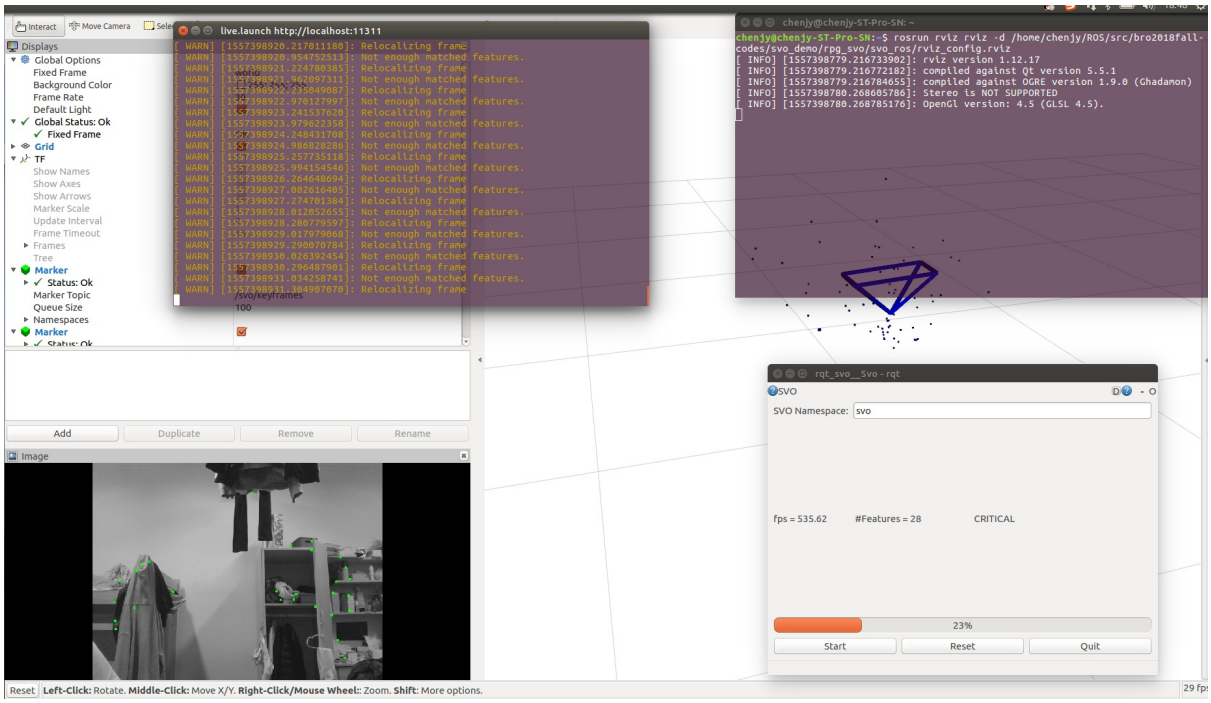
```
$ cd ~/catkin_USB_cam  #Scatkin_USB_cam为USB_camera的工作空间
$ source devel/setup.bash  #刷新环境
$ roslaunch usb_cam usb_cam-test.launch
```

启动SVO

```
#打开新终端
$ cd  ~/SVO_demo/  #SVO_demo为SVO的工作空间
$ source devel/setup.bash  #刷新环境
$ roslaunch svo_ros live.launch
#打开新终端
$ cd  ~/SVO_demo/  #SVO_demo为SVO的工作空间
$ source devel/setup.bash  #刷新环境
$ rosrun rviz rviz -d ~/SVO_demo/src/rpg_svo/svo_ros/rviz_config.rviz
```

```
# 打开新终端
$ cd  ~/SVO_demo/  #SVO_demo为SVO的工作空间
$ source devel/setup.bash  #刷新环境
$ rosrun rqt_svo rqt_svo # SVO NameSpace栏输入svo
```

操作截图如下：



2.4 SVO实际部署--运行SVO+Xbot

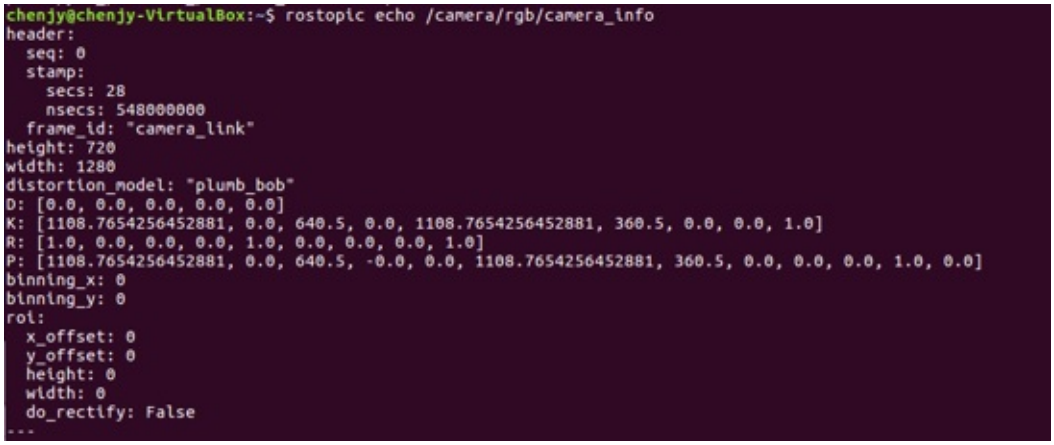
2.4.1 安装Xbot仿真机器人

```
#下载源码
$ mkdir -p tutorial_ws/src   #创建工作空间
$ cd tutorial_ws/src         #进入src路径
$ git clone https://github.com/DroidAITech/ROS-Academy-for-Beginners.git  #下载源码
# 安装依赖
$ cd ~/tutorial_ws
$ rosdep install --from-paths src --ignore-src --rosdistro=kinetic –y
# 编译
$ cd ~/tutorial_ws
$ catkin_make
```

2.4.2 SVO配置参数修改

- 修改相机内参文件
  - 修改rpg_svo/svo_ros/param/camera_atan.yaml文件
    1.启动X波特机器人后，通过打印相机信息的topic获得相关参数

    `$ rostopic echo /camera/rgb/camera_info` 截图如下：



2.修改rpg_svo/svo_ros//param/camera_atan.yaml文件参数为标定得到的对应参数

```
* 修改~/ROS/src/bro2018fall-codes/svo_demo/rpg_svo/svo_ros/launch/live.launch文件第6行为：<param name="cam_topic"
value="/camera/rgb/image_raw" type="str" />
```

2.4.3 启动SVO+Xbot

启动Xbot机器人

```
#打开新终端
$ cd ~/tutorial_ws          #进入Xbot工作空间
$ source devel/setup.bash   #刷新环境
$ roslaunch robot_sim_demo robot_spawn.launch
```

启动SVO

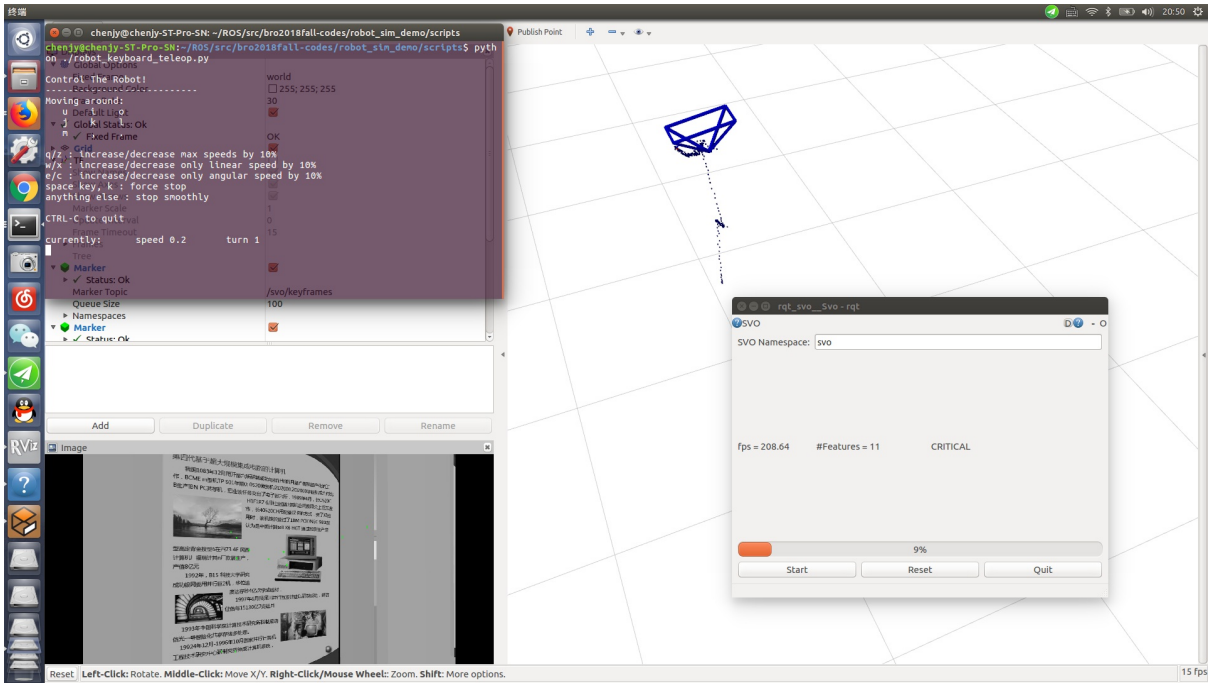
```
#打开新终端
$ cd  ~/SVO_demo/  #SVO_demo为SVO的工作空间
```

```
$ source devel/setup.bash  #刷新环境
$ roslaunch svo_ros live.launch
#打开新终端
$ cd  ~/SVO_demo/  #SVO_demo为SVO的工作空间
$ source devel/setup.bash  #刷新环境
$ rosrun rviz rviz -d ~/SVO_demo/src/rpg_svo/svo_ros/rviz_config.rviz
# 打开新终端
$ cd  ~/SVO_demo/  #SVO_demo为SVO的工作空间
$ source devel/setup.bash  #刷新环境
$ rosrun rqt_svo rqt_svo # SVO NameSpace栏输入svo
```

控制机器人移动，测试算法效果

```
#打开新终端
$ cd ~/tutorial_ws          #进入Xbot工作空间
$ source devel/setup.bash   #刷新环境
$ rosrun robot_sim_demo robot_keyboard_teleop.py
```

操作截图如下：



2.4.4 SVO探索总结

1.SVO是一个VO算法，没有闭环检测部分，这意味着丢失后没法重定位；
2.SVO代码伪开源，开源的代码实际部署效果较差，不实用；
3.SVO使用直接法所以就有直接法的所有缺点：对光照及运动的鲁棒性不足；
4.速度快，在PC平台上能达到每秒400多帧；
5.论文及代码清晰易读，适合VSALM的学习；

LSD

本章介绍的算法是J.Engle等人于2014年提出的SLAM工作，标志着单目直接法在SLAM中的成功应用。其核心贡献是将直接法应用到半稠密的单目SLAM中。

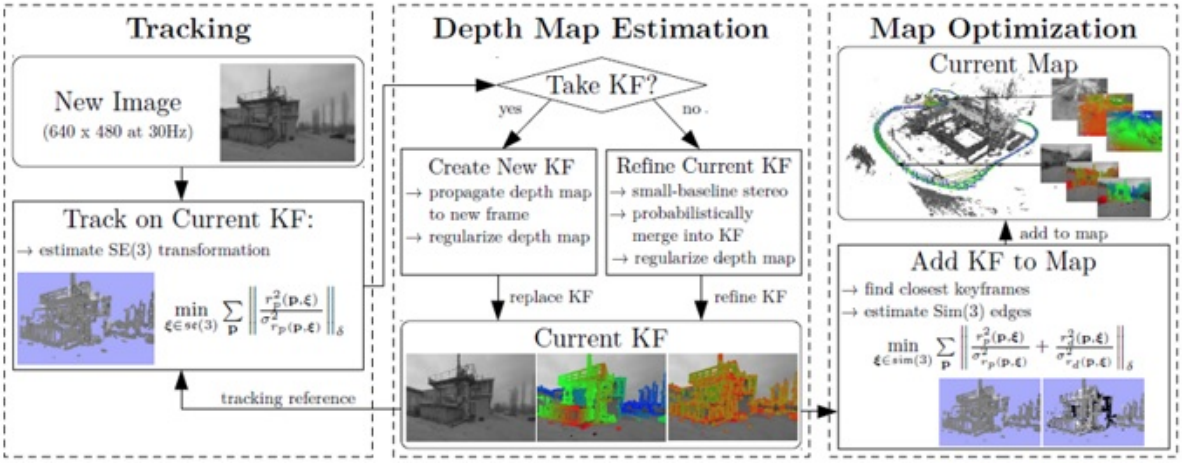
本章首先对此算法的理论进行了简单的介绍，然后同前面一样，分别介绍了利用现有的公开数据集以及笔记本摄像头对算法进行测试的步骤，最后在Xbot机器人部署了算法并实地进行了测试。

3.1 理论知识

3.1.1 LSD简介

LSD (Large-Scale Direct Monocular SLAM)是J. Engle等人于2014年提出的SLAM工作。类比于ORB-SLAM之于特征点，LSD-SLAM则标志着单目直接法在SLAM中的成功应用。LSD-SLAM的核心贡献是将直接法应用到了半稠密的单目SLAM中。它不仅不需要计算特征点，还能构建半稠密的地图。在特征点稀少的环境下仍能达到很高的定位精度与鲁棒性。

LSD-SLAM系统整个过程分为三大模块，第一个模块是Tracking,第二个模块是Depth Map Estimation, 第三个模块是Map Optimization



3.1.2 Tracking

图像跟踪：连续跟踪从相机获取到的新“图像帧”。也就是说用前一帧图像帧作为初始姿态，估算出当前参考关键帧和新图像帧之间的刚体变换 。当一个新的图像(或者说新的观测)输入时，帧间的位姿变换由关键帧和当前图像的灰度差优化得到。通过优化算出当前观测到的图像到关键帧的位姿变换，从而得到当前的位置姿态信息。

3.1.3 Depth Map Estimation

Tracking成功之后，就进入深度估计过程。首先是判断，是不是新建关键帧。关键帧的创建是根据相机移动来判断的，如果相机移动了足够远，那么就重新创建一个关键帧，否则对当前关键帧进行深度更新。

3.1.4 Map Optimization

地图优化：长距离会出现尺度漂移，因此需要做闭环检测和全局优化。首先需要插图关键帧到地图当中。如何去寻找要插入的位置呢？首先去寻找所有可能相似的关键帧，并计算视觉意义上的相似度，之后对这些帧进行排序，得到最相似的那几帧，某一帧的相似度足够高，那么便将这一帧插入Map，然后执行图优化。

3.2 LSD实际部署

3.2.1 运行环境

部署环境介绍

- Ubuntu 16.04
- ROS-kinetic 安装教程
- Xbot仿真平台 部署教程

3.2.2 安装部署

- **安装LSD**

```
$ mkdir -p ~/catkin_lsd/src
$ cd ~/catkin_lsdws/src/
# 安装依赖及配置
$ sudo apt-get install ros-kinetic-libg2o ros-kinetic-cv-bridge liblapack-dev libblas-dev freeglut3-dev libsuit
esparse-dev libx11-dev
$ sudo apt install libqglviewer-dev-qt4
$ cd /usr/lib/x86_64-linux-gnusudo
$ ln -s libQGLViewer-qt4.so libQGLViewer.so
# 下载源码(https://yt.droid.ac.cn/xbot-u/vslam-evaluation/tree/master/LSD) 到~/catkin_lsd/src/目录下
```

- **编译LSD**

```
$ cd  ~/catkin_lsd
$ catkin_make
```

- **下载数据集**

  $ cd ~/catkin_lsdws/src/ 下载数据集并解压缩到~/catkin_lsdws/src/目录下 链接：vmcremers8.informatik.tu-muenchen.de/lsd/LSD_room.bag.zip

3.2.3 运行LSD及数据集

- **启动LSD**
- 启动LSD-viewer查看点云

  ```
  $ cd  ~/catkin_lsd
  $ source devel/setup.bash
  $ rosrun lsd_slam_viewer viewer
  ```

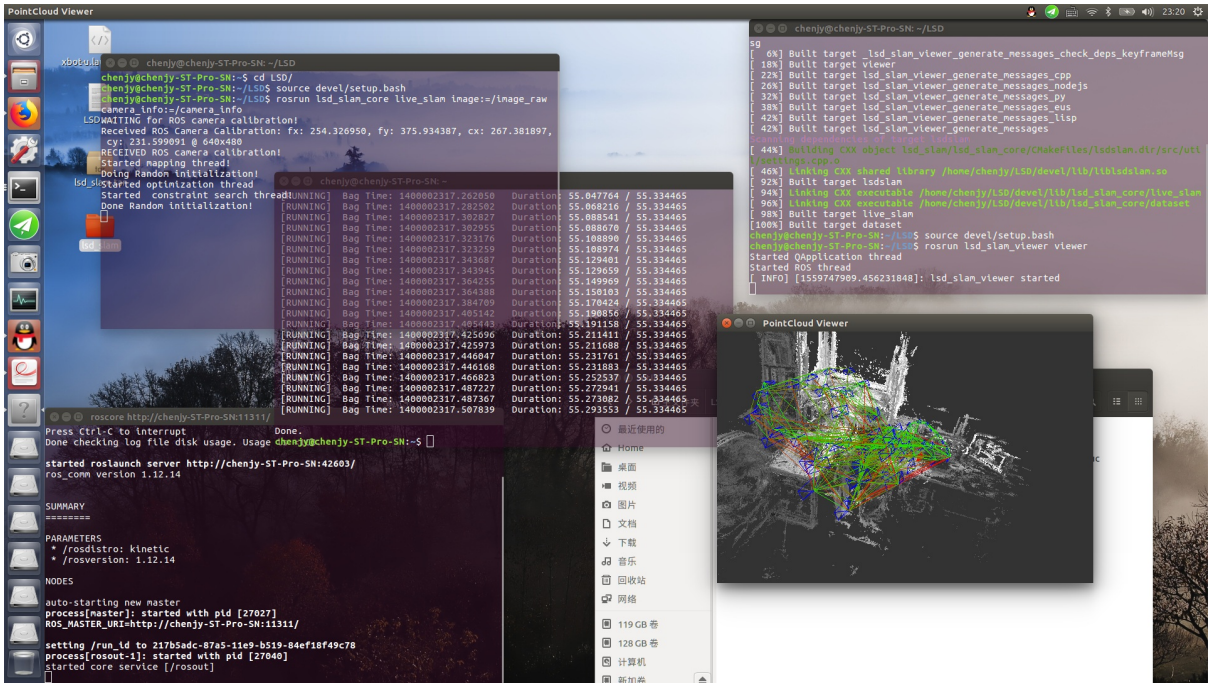
  - 启动LSD-core

    ```
    $ cd  ~/catkin_lsd
    $  source devel/setup.bash
    $ rosrun lsd_slam_core live_slam image:=/image_raw camera_info:=/camera_info
    ```

- **播放rosbag**

下载的数据集所在文件夹打开终端：

$ rosbag play ./LSD_room.bag 运行截图如下：



3.3LSD实际部署--运行LSD+USB_camera

3.3.1 安装摄像头驱动usb_cam

```
# 打开新终端
$ mkdir -p ~/catkin_USB_cam/src  #创建USB_cam工作空间catkin_USB_cam
$ cd  ~/catkin_USB_cam/src
$ git clone https://github.com/ros-drivers/usb_cam.git  #下载源码
$ cd ..
$ catkin_make  #编译
```

3.3.2 相机标定

- 打印一张标定板
- 打开相机

  ```
  # 打开新终端
  $ cd ~/catkin_USB_cam  #Scatkin_USB_cam为USB_camera的工作空间
  $ source devel/setup.bash  #刷新环境
  $ roslaunch usb_cam usb_cam-test.launch
  ```

- 打开矫正窗口

  ```
  $ rosrun camera_calibration cameracalibrator.py --size 8x6 --square 0.0245 image:=/usb_cam/image_raw camera
  :=/usb_cam
  ```

- 移动标定板
  不断移动标定板，直到CALIBRATE按钮变亮
- 计算矫正参数
  点击CALIBRATE按钮就会进行标定（标定过程将持续一两分钟）。并可以在命令窗中看到标定参数，点击COMMIT将结果保存到电脑。

3.3.3 修改配置参数

修改 `~/catkin_lsd/src//lsd_slam/lsd_slam_core/calib/pinhole_example_calib.cfg` 文件参数,文件参数意义如下：

```
fx/width fy/height cx/width cy/height 0
width height
none
width height
```

3.3.4 启动SVO+USB_camera

启动USB_camera

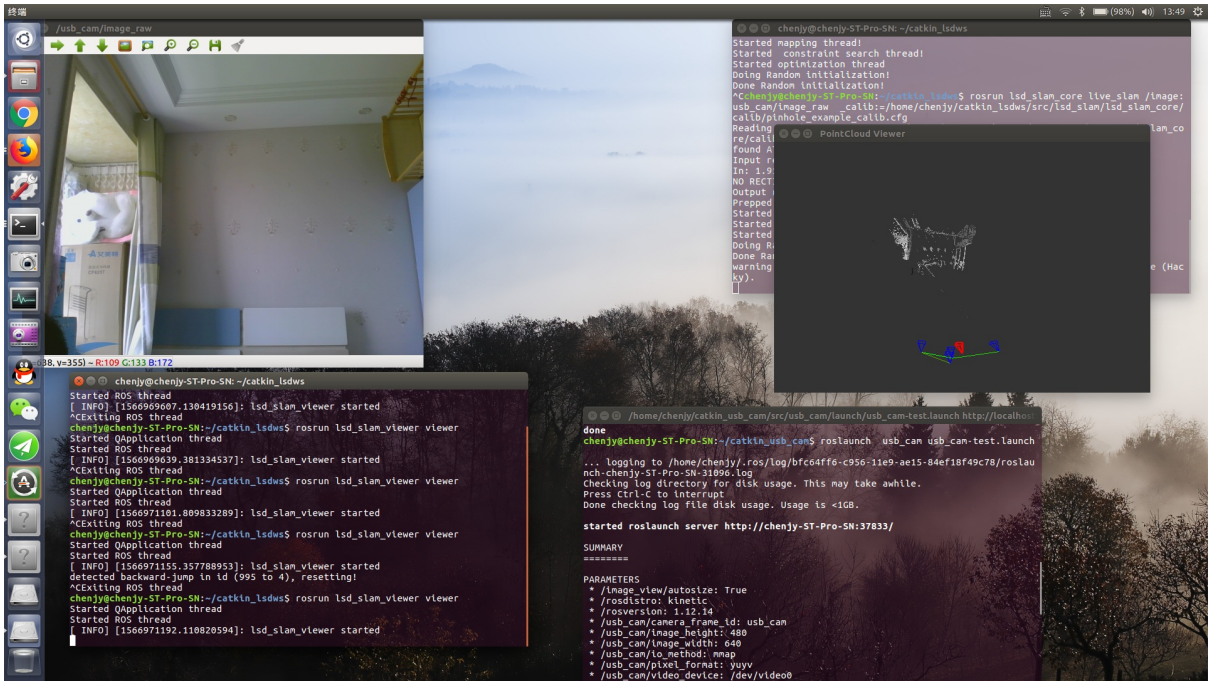
```
$ cd ~/catkin_USB_cam  #Scatkin_USB_cam为USB_camera的工作空间
$ source devel/setup.bash  #刷新环境
$ roslaunch usb_cam usb_cam-test.launch
```

启动LSD

```
#打开新终端
$ cd  ~/catkin_lsd
$ source devel/setup.bash
$ rosrun lsd_slam_viewer viewer
#打开新终端
```

```
$ cd  ~/catkin_lsd  #catkin_lsd为LSD的工作空间
$ source devel/setup.bash  #刷新环境
$ rosrun lsd_slam_core live_slam /image:=/usb_cam/image_raw  _calib:=~/catkin_lsd/src/lsd_slam/lsd_slam_core/ca
lib/pinhole_example_calib.cfg
```

操作截图如下：



3.4 LSD实际部署--运行LSD+Xbot

3.4.1 安装Xbot仿真机器人

```
#下载源码
$ mkdir -p tutorial_ws/src   #创建工作空间
$ cd tutorial_ws/src         #进入src路径
$ git clone https://github.com/DroidAITech/ROS-Academy-for-Beginners.git  #下载源码
# 安装依赖
$ cd ~/tutorial_ws
$ rosdep install --from-paths src --ignore-src --rosdistro=kinetic –y
# 编译
$ cd ~/tutorial_ws
$ catkin_make
```

3.4.2 启动LSD+Xbot

启动Xbot机器人

```
#打开新终端
$ cd ~/tutorial_ws         #进入Xbot工作空间
$ source devel/setup.bash   #刷新环境
$ roslaunch robot_sim_demo robot_spawn.launch
```

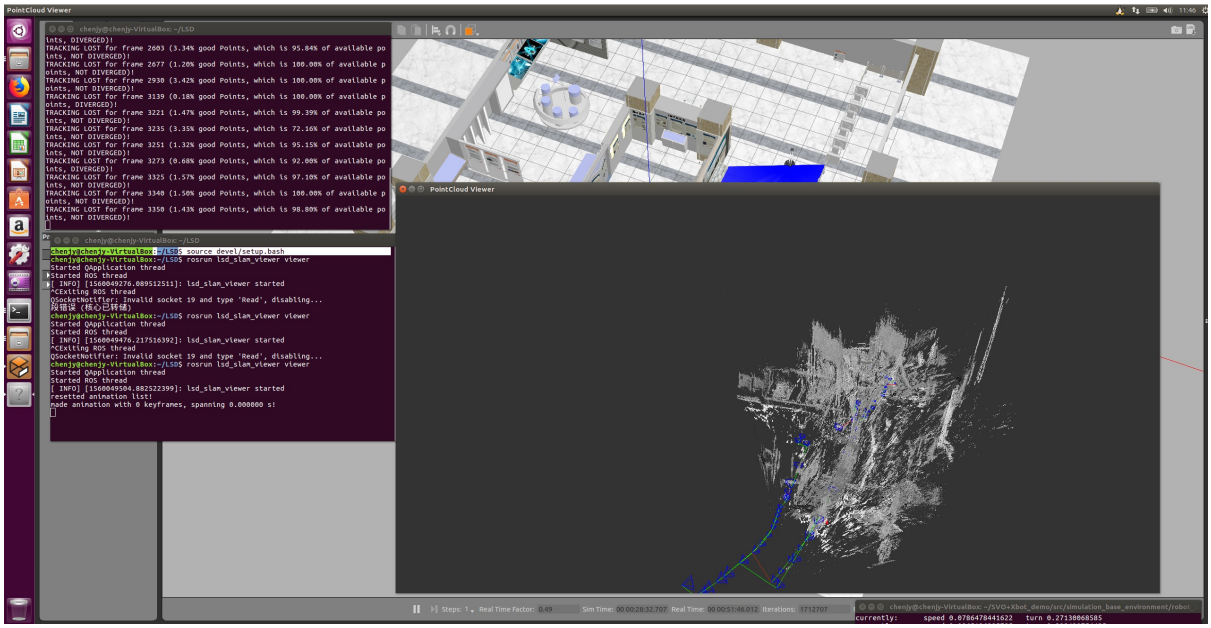
启动LSD

```
#打开新终端
cd catkin_lsd #catkin_lsdws为LSD工作空间
source devel/setup.bash
rosrun lsd_slam_core live_slam /image:=/camera/rgb/image_raw  /camera_info:=/camera/rgb/camera_info
#打开新终端
cd catkin_lsd #catkin_lsdws为LSD工作空间
source devel/setup.bash
rosrun lsd_slam_viewer viewer
```

控制机器人移动，测试算法效果

```
#打开新终端
$ cd ~/tutorial_ws         #进入Xbot工作空间
$ source devel/setup.bash   #刷新环境
$ rosrun robot_sim_demo robot_keyboard_teleop.py
```

操作截图如下：



3.4.3 LSD探索总结

在CPU上实现了实时半稠密场景的重建；
对相机内参和曝光非常敏感，在相机快速运动时容易丢失；
没有基于直接法的回环检测。因此必须依赖于特征点方法来进行回环检测，尚未摆脱特征点的计算；

ORB

不同于前面介绍的VSLAM算法，本章介绍的ORB-SLAM和ORB-SLAM2是基于特征点法的VSLAM算法，ORB-SLAM支持单目，而ORB-SLAM2支持单目、双目和RGBD接口。

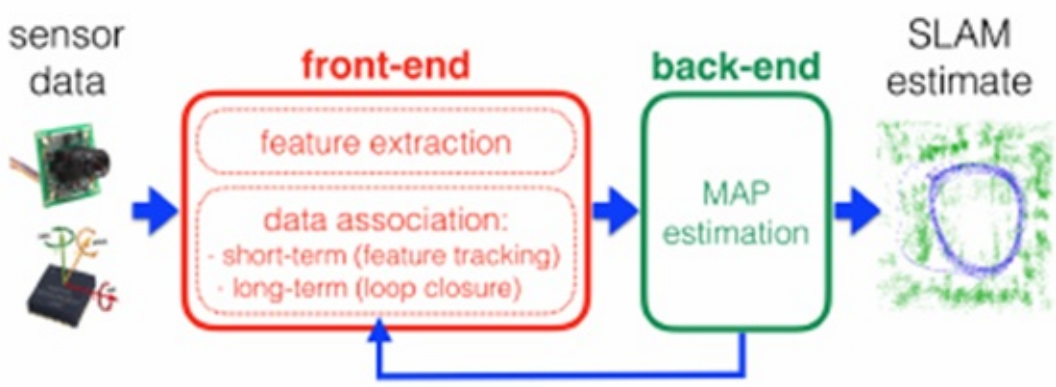
本章分为五个部分来介绍，首先是理论知识介绍，其次是讲解了如何利用公开数据集来运行ORB和ORB2,再介绍利用实际摄像头测试ORB的流程以及仿真实验，最后对代码流程进行了简单的阐述。

4.1 ORB-SLAM & ORB-SLAM2 理论介绍

4.1.1 Introduction

ORB-SLAM是2015年, 西班牙萨拉戈萨大学*(Universidad de Zaragoza)*Raul Mur Artal等发表在“Transactions on Robotics"期刊的一篇文章，并在GitHub上开源了算法。可以说ORB-SLAM，是对视觉SLAM的一个阶段性的突破，通过使用计算量小的ORB特征和三个并行线程——追踪、建图和重定位，ORB-SLAM在单目视觉SLAM各方面都取得了较为完善的突破。

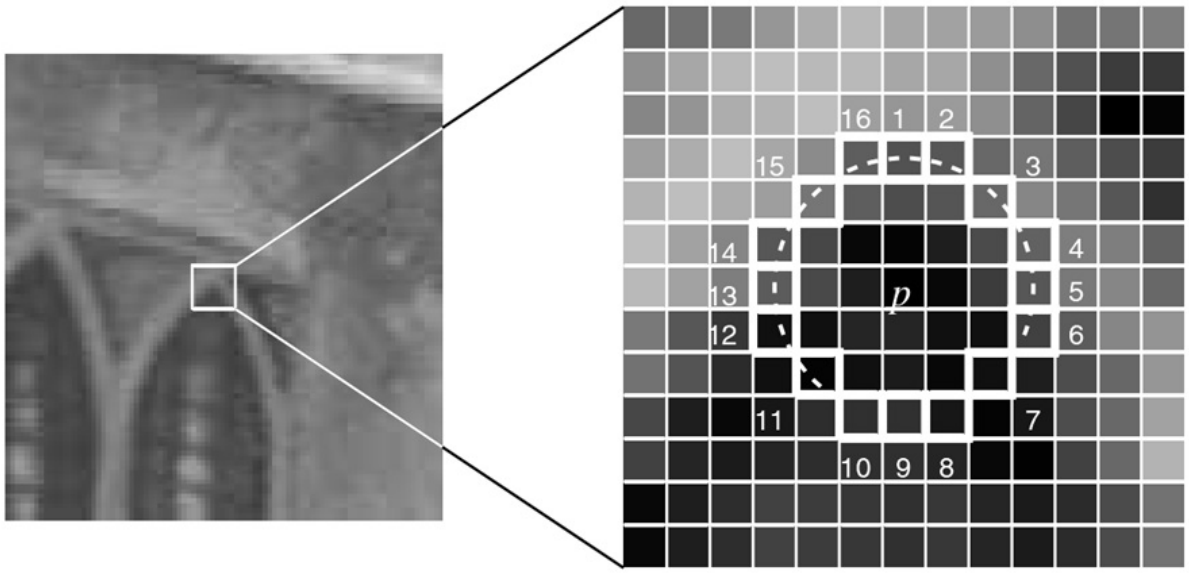
ORB-SLAM是基于特征点的单目实时SLAM方案，也就是使用了ORB特征点的方法，获取一张图片上的特征点有多种方法，像SIFT, SURF等。对于特征点法而言，特征方法直接影响到了系统的前端稳定性和实时性，一个典型的V-SLAM由这几部分组成——传感器、前端、后端和回环检测。



4.1.2 ORB特征

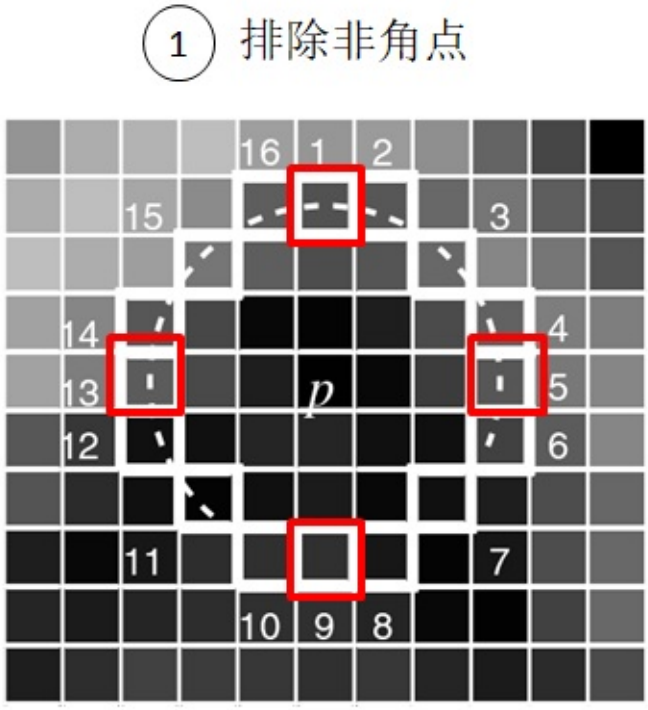
ORB全称是 Oriented FAST and Rotated BRIEF，也就是带有方向的FAST关键点和有旋转性的BRIEF描述子组合而成，其中FAST又是一种检测图像关键点的方法，全称是Featurs from Accelerated Segment Test,FAST关键点检测方式简单快捷，具体的方式为在一个候选关键点为圆心的像素周边做一个半径为r的像素圆，判断其圆弧画到的点是否有超过连续n个点的像素值明显比候选关键点亮或者暗。

FAST关键点



具体的判断步骤是

1. 排除明显不是关键点的候选者，通过检测罗盘的东西南北四个像素点的亮度值，如果有大于等于3个点能够较为明显亮于或者暗于候选点，才进行下一步检测，否则判断为非关键点；



$$I_{p\to x} \geq I_p + t \bigcup I_{p\to x} \leq I_p + t, x = 1,5,9,13$$

2. 将上一步的点，判断连续的n个像素值是否能够亮于或者暗于候选点，是则可能为关键点，否则排除

然后选出的候选点会有很多，需要解决的问题有候选点“扎堆”的情况，解决方法是通过三个原则来判断是否能成为关键点

- 圆弧半径上，n值最大者保留为角点；
- 圆弧半径上，阈值t最大者保留为角点；
- 圆弧半径上，各连续像素块灰度值和中心灰度值的绝对值差最大者保留为角点。

最后一个原则可以数值化表达为

$$V = \max\left(\sum_{x\in S_{\text{bright}}} |I_{p\to x} - I_p| - t \ , \ \sum_{x\in S_{\text{dark}}} |I_p - I_{p\to x}| - t\right)$$

$$S_{\text{bright}} = \{x | I_{p \to x} \geq I_p + t\}$$
$$S_{\text{dark}} = \{x | I_{p \to x} \leq I_p - t\}$$

其中V较大者成为关键点，舍弃临近的V值较小者。选取三个原则中的前N个候选点，既可成为关键点。

BRIEF描述子

在关键点p附近，取一块S*S的邻域，然后以某种模式选取n对点(128,256,512)，比较没对点的亮度值，前者亮则为1，后者亮则为0，将这些数字排列成一个组数组，可以得到二进制向量，从而简化描述子的描述方式。

$$\tau(\mathbf{p}; \mathbf{x}, \mathbf{y}) := \begin{cases} 1 & \text{if } \mathbf{p}(\mathbf{x}) < \mathbf{p}(\mathbf{y}) \\ 0 & \text{otherwise} \end{cases}$$

$$f_{n_d}(\mathbf{p}) := \sum_{1 \leq i \leq n_d} 2^{i-1} \tau(\mathbf{p}; \mathbf{x}_i, \mathbf{y}_i)$$

符号示意

τ：对比结果

p：$S * S$的像素块

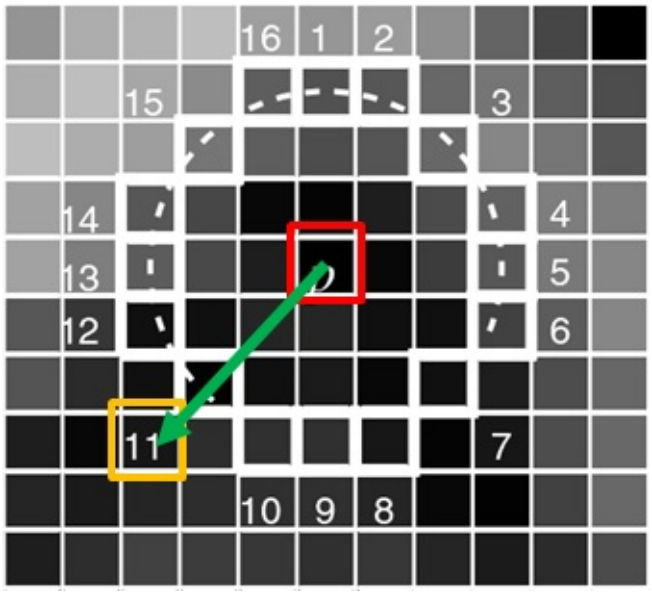
$x = [u \quad v]^T$：像素位置

$p(x)$：像素灰度值

n_d：取点对数/字符串维度

oFAST

Oriented FAST是改进后的FAST关键点，增加了FAST关键点的方向性，从而可以通过描述子，得到一定的方向不变性，增强特征点的鲁棒性。



计算带权重的像素和

$$m_{pq} = \sum_{x,y} x^p y^q I(x,y)$$

计算像素质心

$$C = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}} \right)$$

计算第一象限角θ

$$\theta = \text{atan2}(m_{01}, m_{10})$$

rBRIEF

有一定旋转抵抗性的BRIEF描述子，通过机器学习，得到一种取点对的方式，能够增加描述子的区别特性，同时有一定的抵抗旋转造成变化的能力。

具体方式为

① 从PASCAL 2006 中建立一个300k的关键点训练集合

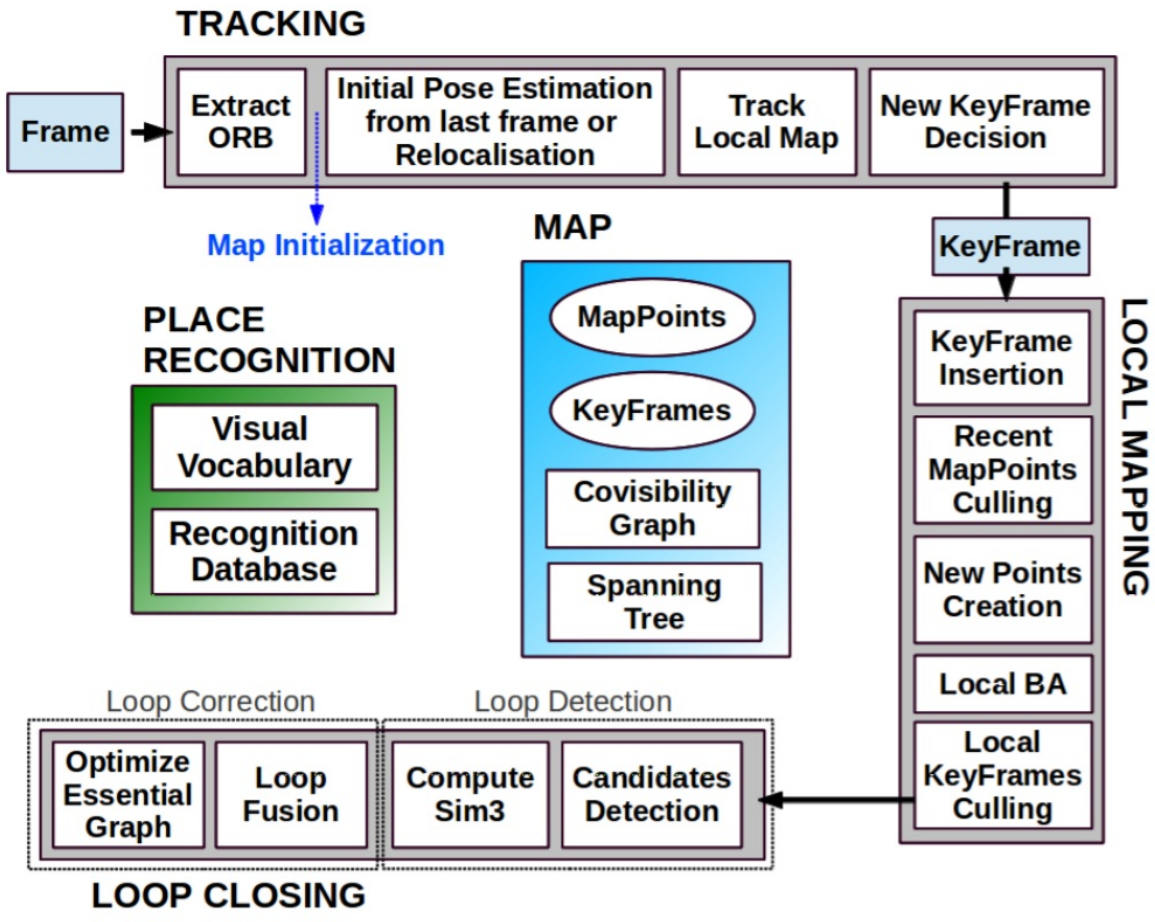
② 每个关键点的31*31领域内提取所有可能的模式

③ 对要比较得到二进制结果的点进行5*5小窗口的取均值，可以得到$M = C_{31-5}^{2}$种可能的模式

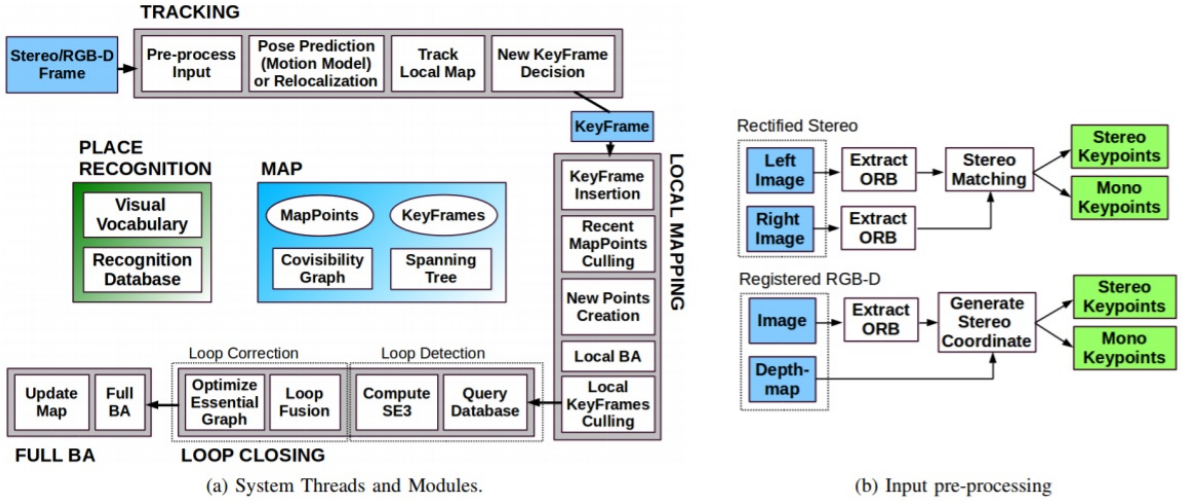
④ 可以得到300k * M 个二进制结果，按照均值和0.5的绝对值差排序成矩阵T

⑤ 贪心法搜索：
1. 将T中第一个向量放入R,并从T中移除；
2. 从T中取出下一个向量，和R中所有的测试二进制比较，如果它的绝对相关性大于某个值，舍弃这个向量，否则将其加入R;
3. 迭代以上步骤，直到有256个向量融入R，如果不够256，降低相关性的阈值。

4.1.3 ORB1&2代码框架



ORB是三线程并行的SLAM系统，包括Tracking, Local Mapping和Loop Closing三个部分，它所生成的地图，包括地图点、关键帧、共视图和生成树。



(a) System Threads and Modules. (b) Input pre-processing

相对于ORB1，ORB2增加了双目和RGB-D相机的支持，也增加了使用两种模式点的方式。

4.2 ORB-SLAM安装编译运行

4.2.1 Introduction

ORB-SLAM源码开源托管于Github,地址是 https://github.com/raulmur/ORB_SLAM ， 可以通过Git clone代码或者通过下载压缩包得到开源程序，由于ORB-SLAM依托于ROS，且对应的版本如下

Ubuntu 12.04	Ubuntu 14.04
Fuerte, Groovy, Hydro	Indigo

现在主流使用的Ubuntu系统为16.04版本，ROS也多为Kinetic，所以修改了原库中部分代码，适应新的系统和ROS框架，修改后的代码位置为：https://github.com/johnchars/ORB_SLAM ，可以按照任意库下的方式进行安装。 4.2.2 依赖项安装

- Boost

```
sudo apt-get install libboost-all-dev
```

Boost 库用于同时启动不同的线程，ORB-SLAM分为三个线程Tracking, Local Mapping and Loop Closing

- ROS

  参考官方wiki

- OpenCV

```
cd && mkdir Repo && cd Repo
git clone https://github.com/opencv/opencv.git
cd opencv
git branch -a
git checkout origin/2.4
git branch -m origin/2.4 opencv-2.4
git branch
mkdir build && cd build
cmake .. && make -j4
sudo make install
```

> 这里也可以使用官网下载，注意选择2.4版本; make -j*中数字可以自己决定

- g2o

```
    sudo apt-get install libeigen3-dev
```

  安装g2o图优化库需要安装eigen库，使用了一个修改过的g2o库，在Thirdparty/下有这个库，不需要下载

- DBow2

  同样的第三库，在Thirdparty/下有这个库，用于回环检测。

4.2.3编译安装

- 创建一个安装位置

```
cd && mkdir -p slam_ws/src && cd slam_ws/src
catkin_init_workspace
cd .. && catkin_make
echo "source ~/slam_ws/devel/setup.bash" >> ~/.bashrc
git clone https://github.com/raulmur/ORB_slam_ws.git ORB_slam_ws
```

- 修改环境变量 ROS_PAKCAGE_PATH，在~/.bashrc末尾增加

> export ROS_PACKAGE_PATH=${ROS_PACKAGE_PATH}:/home/xbot/slam_ws/src

- 编译g2o

```
cd ~/slam_ws/src/ORB_SLAM/Thirdparty/g2o/
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j4
```

- 编译DBoW2

```
cd ../../DBoW2
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j4
```

- 编译ORB_SLAM

删除manifest.xml 中的 `<depend package="opencv2"/>`

在ORBextractor.cc中添加头文件，路径是/home/xbot/slam_ws/src/ORB_SLAM/src

```
#include <opencv2/features2d/features2d.hpp>

#include <opencv2/imgproc/imgproc.hpp>
```

在/home/xbot/slam_ws/Thirdparty/g2o/g2o/solvers中的linear_solver_eigen.h中修改

```
56: typedef Eigen::PermutationMatrix<Eigen::Dynamic, Eigen::Dynamic, SparseMatrix::Index> PermutationMatrix;
修改为
56: typedef Eigen::PermutationMatrix<Eigen::Dynamic, Eigen::Dynamic> PermutationMatrix;
```

在/home/xbot/slam_ws/src/ORB_slam_ws下修改CMakeLists.txt文件，增加

```
find_package(Boost COMPONENTS system)

include_directories(
${Boost_INCLUDE_DIRS} // adding this line
)
target_link_libraries(${PROJECT_NAME}
${Boost_LIBRARIES}
)
```

开始编译...

```
cd ~/slam_ws/src/ORB_SLAM/
mkdir build && cd build
cmake .. -DROS_BUILD_TYPE=Release
make -j4
编译成功显示
Build type: RelWithDebInfo
-- Boost version: 1.58.0
-- Found the following Boost libraries:
--   system
-- Configuring done
-- Generating done
-- Build files have been written to: /home/xbot/slam_ws/src/ORB_SLAM/build
```

```
[  0%] Built target rospack_genmsg_libexe
[  0%] Built target rosbuild_precompile
[  5%] Linking CXX executable ../bin/ORB_SLAM
[100%] Built target ORB_SLAM
```

1. 可能的编译错误解决

2. [x] 'FAST' was not declared in this scope FAST(cellImage, cellKeyPoints [i] [j], fastTh, true);

> 解决方法GitHub #44
>
> 在src/ORBextractor.cc中增加
>
> ```
> #include <opencv2/opencv.hpp>
> ```
>
> 增加在#include 前

- [x] DSO missing from command line

> 解决方Github#552
>
> 在CMakeLists.txt中增加Boost头文件
>
> ```
> include_directories(${Boost_INCLUDE_DIRS} #增加这一行
> ```

- [x] cmake Configuring incomplete, errors occurred!

  解决方法，检查ROS_PACKAGE_PATH路径，在.bashrc文件下

- [ ] 其他问题可以在ORB_SLAM下检索关键词

4.2.4 运行算法

a. 解压Data下的词库和设置文件

```
cd /home/xbot/slam_ws/src/ORB_SLAM/Data
xbot@nuc:~/slam_ws/src/ORB_SLAM/Data$ tar xzvf ORBvoc.txt.tar.gz
ls
增加 ORBvoc.txt文件
```

b.逐个文件启动

```
Ctrl+Alt+t
roscore
Ctrl+Alt+t
cd /home/xbot/slam_ws/src/ORB_SLAM
rosrun ORB_SLAM ORB_SLAM Data/ORBvoc.txt Data/Setting.yaml
```

[todo] adding picture orb_node_success

```
Ctrl+Alt+t
rosrun image_view image_view image:=/ORB_SLAM/Frame _autosize:=true
```

[todo] adding picture

```
cd /home/xbot/slam_ws/src/ORB_SLAM
rosrun rviz rviz -d Data/rviz.rviz
```

[todo] adding picture orb_rviz_d

c.使用launch文件启动

```
cd /home/xbot/slam_ws/src/ORB_SLAM
roslaunch ExampleGroovyOrNewer.launch
```

d. 使用Example.bag 检测安装是否成功

- 下载bag文件
- 切换到bag文件路径，这里在/home/xbot/下建立了Dataset/文件夹，并将文件放到这里

```
cd /home/xbot/Dataset/
tar xzvf Example.bag.tar.gz
rosbag play --pause Example.bag #按空格开始
```

4.3 ORB-SLAM2 安装编译运行

4.3.1 Introduction

ORB-SLAM2源码开源托管于Github,地址是 : https://github.com/raulmur/ORB_SLAM2 ，可以通过Git克隆代码或者通过下载压缩包得到开源程序。区别于ORB-SLAM1，ORB-SLAM2可以直接脱离ROS编译使用，同样提供可以支持脚本安装的方式，代码位置：https://github.com/johnchars/ORB_SLAM2 。

4.3.2 依赖项安装

- Pangolin

```
git clone https://github.com/stevenlovegrove/Pangolin.git
sudo apt-get install libglew-dev
sudo apt-get install cmake
sudo apt-get install libpython2.7-dev
cd Pangolin/ && mkdir build && cd build/
cmake ..
cmake --build .
echo "Pangolin install finished"
```

- OpenCV

```
cd ../..
git clone https://github.com/opencv/opencv.git
sudo apt-get install build-essential
sudo apt-get install cmake git libgtk2.0-dev pkg-config libavcodec-dev libavformat-dev libswscale-dev
sudo apt-get install python-dev python-numpy libtbb2 libtbb-dev libjpeg-dev libpng-dev libtiff-dev libjasper-de
v libdc1394-22-dev
cd opencv/ && mkdir build && cd build/
cmake -D CMAKE_BUILD_TYPE=Release -D CMAKE_INSTALL_PREFIX=/usr/local ..
make -j4
sudo make install
```

- Eigen

```
cd ../../
git clone https://github.com/eigenteam/eigen-git-mirror.git Eigen
cd Eigen/
git branch -v
git checkout 3.3.6
mkdir build && cd build
cmake ..
sudo make install
```

4.3.3 编译安装

```
echo "Configuring and building Thirdparty/DBoW2 ..."

cd ../Thirdparty/DBoW2
mkdir build
cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j4

cd ../../g2o

echo "Configuring and building Thirdparty/g2o ..."
```

```
mkdir build
cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j4

cd ../../../

echo "Uncompress vocabulary ..."

cd Vocabulary
tar -xf ORBvoc.txt.tar.gz
cd ..

echo "Configuring and building ORB_SLAM2 ..."

mkdir build
cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j4
```

4.3.4 编译ROS节点

```
cd ../Examples/ROS
echo "export ROS_PACKAGE_PATH=${ROS_PACKAGE_PATH}:HOME}/slam_ws/src/ORB2_CWJ/Examples/ROS"\
>> ~/.bashrc
source ~/.bashrc
./build_ros.sh
```

错误说明

直接编译raulmur/ORB_SLAM2的package可能会出现的两个错误

- 'usleep' not declared

在Examples/Monocular/mono_euroc.cc mono_tum.cc, Examples/RGB-D/rgbd_tum.cc, Examples/Stereo/stereo_euroc.cc stereo_kitti.cc 及 src/ LocalMapping.cc LoopClosing.cc, System.cc

Tracking.cc Viewer.cc中增加头文件

`#include <unistd.h>`

- '_ZN5boost6system15system_categoryEv' 和 DSO missing from command line

在Examples/ROS/ORB_SLAM2/CMakeLists.txt中修改如下

```
set(LIBS
${OpenCV_LIBS}
${EIGEN3_LIBS}
${Pangolin_LIBRARIES}
${PROJECT_SOURCE_DIR}/../../../Thirdparty/DBoW2/lib/libDBoW2.so
${PROJECT_SOURCE_DIR}/../../../Thirdparty/g2o/lib/libg2o.so
${PROJECT_SOURCE_DIR}/../../../lib/libORB_SLAM2.so
-lboost_system #adding this line
)
```

4.3.5 d415_orb

这个package集合了标定文件和launch文件，简化了使用过程，通过配合使用这个package，可以省去标定D415相机过程。

4.3.6 数据集检验运行

- https://vision.in.tum.de/data/datasets/rgbd-dataset/download `TUM`
- http://www.cvlibs.net/datasets/kitti/eval_odometry.php `KITTI`
- https://projects.asl.ethz.ch/datasets/doku.php?id=kmavvisualinertialdatasets `EuRoc`

4.4 ORB-SLAM & ORB-SLAM2 仿真实验及实际摄像头实验

4.4.1 Introduction

安装完成ORB-SLAM1 & ORB-SLAM2后，需要对其的性能进行评价，同时也需要知道如何运用到实际场景下，通过仿真及真实摄像头下的操作，可以直观感受到ORB-SLAM的优缺点。

仿真实验采用 https://github.com/DroidAITech/ROS-Academy-for-Beginners ，库下的仿真实例，摄像头将以普通的单目摄像头如笔记本的摄像头和Realsense D415摄像头作为例子。

4.4.2 仿真实验

- 按照 https://github.com/DroidAITech/ROS-Academy-for-Beginners 下的步骤，克隆到本地，并运行验证是否安装成功

```
cd && mkdir -p catkin_ws/src
cd ~/catkin_ws/src
git clone https://github.com/DroidAITech/ROS-Academy-for-Beginners.git
cd ~/catkin_ws
rosdep install --from-paths src --ignore-src --rosdistro=kinetic -y
catkin_make
#每次运行仿真前，需要运行下面的命令刷新环境
source ~/catkin_ws/devel/setup.bash
roslaunch robot_sim_demo robot_spawn.launch
```

- 修改topic名称，适应于ORB1&2
  - ORB-SLAM1对应的topic为*/camera/image_raw*
  - ORB-SLAM2对应的topic为*/camera/image_raw*, */camera/left/image_raw* & */camera/right/image_raw*, */camera/rgb/image_raw/* & */camera/depth_registered/image_raw/*。

在仿真包 `robot_sim_demo` 的urdf/xbot-u.gazebo中，有单目摄像头的信息

```
<gazebo reference="camera_link">
      <sensor type="depth" name="camera">
        <plugin name="kinect_camera_controller" filename="libgazebo_ros_openni_kinect.so">
          <cameraName>camera</cameraName>
          <alwaysOn>true</alwaysOn>
          <updateRate>10</updateRate>
          <imageTopicName>rgb/image_raw</imageTopicName>
          <depthImageTopicName>depth/image_raw</depthImageTopicName>
          <pointCloudTopicName>depth/points</pointCloudTopicName>
          <cameraInfoTopicName>rgb/camera_info</cameraInfoTopicName>
     <depthImageCameraInfoTopicName>depth/camera_info</depthImageCameraInfoTopicName>
          <frameName>camera_link_optical</frameName>
          <baseline>0.1</baseline>
          <distortion_k1>0.0</distortion_k1>
           <distortion_k2>0.0</distortion_k2>
           <distortion_k3>0.0</distortion_k3>
           <distortion_t1>0.0</distortion_t1>
           <distortion_t2>0.0</distortion_t2>
           <pointCloudCutoff>0.4</pointCloudCutoff>
          </plugin>
        </sensor>
</gazebo>
```

可以看到topic的名字是 `rgb/image_raw` ,修改名称为 `image_raw` 即可

- 仿真摄像头Setting.yaml文件配置

```
#camera calibration and distortion parameters (OpenCV)
```

```
Camera.fx: 277.19135641132203
Camera.fy: 277.19135641132203
Camera.cx: 160.5
Camera.cy: 120.5

Camera.k1: 0.0
Camera.k2: 0.0
Camera.p1: 0.0
Camera.p2: 0.0

Camera.width: 320
Camera.height: 240

#Camera frames per second
Camera.fps: 10.0
```

在ORB_SLAM/Data下提供了配置好的museum_sim.yaml文件，也可以自己尝试配置。

- 依顺序启动robot_sim_demo & ORB-SLAM1

```
cd && cd catkin_ws/
source ./devel/setup.bash
roslaunch robot_sim_demo robot_spawn.launch
# Ctrl + Alt + T
cd ~/slam_ws
source ./devel/setup.bash
roslaunch ORB_SLAM museum_sim.launch
```

- 使用键盘控制，用ORB-SLAM1建立地图

```
cd ~/catkin_ws
source ./devel/setup.bash
rosrun robot_sim_demo robot_keyboard_teleop.py
```

按照桌面提示，可以建立对应的地图，结果保存到KeyFrameTrajectory.txt

- ORB—SLAM2的仿真实验十分相似，留给读者尝试(提示：可以通过ORB-SLAM2/launch/mono.launch，写一个仿真的launch文件，注意替换setting.yaml文件)。

4.4.3 数据集使用

- 常用数据集
  - https://vision.in.tum.de/data/datasets/rgbd-dataset
  - http://www.cvlibs.net/datasets/kitti/eval_odometry.php
  - https://projects.asl.ethz.ch/datasets/doku.php?id=kmavvisualinertialdatasets

TUM包括多个序列，下载比较容易，KITTI最小的grayscale，有22个G的大小，是21个序列的合集，EuRoc包含多个序列，下载也比较容易。

- TUM数据集——单目

以fr1/xyz为例，首先下载这个数据集和https://vision.in.tum.de/data/datasets/rgbd-dataset/download中的useful_tools.

对于ORB_SLAM1,需要将下载的序列，转化成bag文件，这里需要https://github.com/raulmur/BagFromImages ，这样一个工具，将图像转化为Bag，使用方法如README.md中介绍，创建了个位置

```
cd ~/slam_ws/src
git clone https://github.com/raulmur/BagFromImages.git BagFromImages

cd BagFromImages
mkdir build
```

```
cd build
cmake ..
make
```

使用方法

```
rosrun BagFromImages BagFromImages 指向image的文件路径 文件扩展名 频率 输出路径/filename.bag
# 例
rosrun BagFromImages BagFromImages ${HOME}/Dataset/TUM/rgbd_dataset_freiburg1_xyz/rgb .png 30 ${HOME}/Dataset/T
UM/fr1_xyz.bag
```

KITTI和EuRoc单目的bag都可以用同样的方式。

对于ORB-SLAM2，不需要借助与ROS就可以进行数据集的使用，如

```
./Examples/Monocular/mono_tum Vocabulary/ORBvoc.txt Examples/Monocular/TUMX.yaml PATH_TO_SEQUENCE_FOLDER
```

详情参照https://github.com/johnchars/ORB_SLAM2

- TUM数据集——RGB-D

RGB-D数据集使用，需要将rgb & depth数据时间对齐，使用TUM中的useful tools，通过

```
python associate.py PATH_TO_SEQUENCE/rgb.txt PATH_TO_SEQUENCE/depth.txt > associations.txt

./Examples/RGB-D/rgbd_tum Vocabulary/ORBvoc.txt Examples/RGB-D/TUMX.yaml PATH_TO_SEQUENCE_FOLDER ASSOCIATIONS_F
ILE
```

例

```
cd {your_path}/useful_tools/

python useful_tools/associate.py rgbd_dataset_freiburg1_xyz/rgb.txt rgbd_dataset_freiburg1_xyz/depth.txt > asso
ciations.txt

./Examples/RGB-D/rgbd_tum Vocabulary/ORBvoc.txt Examples/RGB-D/TUM1.yaml  ASSOCIATIONS_FILE
```

- 误差估计
  - ATE(Absolute Trajectory Error)：通过时间戳融合ground truth 和 estimated trajectory，计算每对位姿间的误差值。
  - RPE(Relative Pose Error)：估计轨迹上的相邻时间戳的一对位姿间的相对位姿变换和ground truth的对应时间戳间相对位姿变换的误差。

4.4 摄像头及实验

1. 笔记本摄像头

http://wiki.ros.org/image_transport/Tutorials ，编写一个发布器，在src/publisher_of_image/中提供了发布对应topic的程序，可以通过将该程序放到slam_ws/src/下，编译运行。

```
rosrun publisher_of_image publisher_of_image NUM
```

*NUM*为设备号，可以通过 `ls /dev/device` 获取，笔记本自带摄像头一般是0，同样的USB摄像头也可以通过这样的方式使用，请读者自行尝试。

1. Intel Realsense D415

2. 首先拥有一个D415的摄像头，如果是其他型号，可以参考修改。对应的内参标定文件，即setting.yaml已经提供，且放到ORB_SLAM/Data/d415_color.yaml & face_cam.yaml。
3. 标定摄像头的内参，如果使用其他摄像头，参考标定http://wiki.ros.org/camera_calibration
4. 安装对应的驱动，并使用rs_camera.launch检验安装是否正确。

安装参考 https://github.com/IntelRealSense/librealsense/blob/master/doc/installation.md, https://github.com/IntelRealSense/librealsense/blob/master/doc/distribution_linux.md, https://github.com/IntelRealSense/realsense-ros 。

- 下载d415_orb，通过对应的launch驱动需要的程序。d415_orb package在GitLab的vslam-evaluation/ORB2下可以找到。

```
# source {workspace}/devel/setup.bash
roslaunch d415_orb rgb-d.launch
#
roslaunch ORB_SLAM2 rgbd.launch
```

附注

- 数据集下载地址
  - https://vision.in.tum.de/data/datasets/rgbd-dataset/download `TUM`
  - http://www.cvlibs.net/datasets/kitti/eval_odometry.php `KITTI`
  - https://projects.asl.ethz.ch/datasets/doku.php?id=kmavvisualinertialdatasets `EuRoc`
- Intel Realsense 驱动安装过程——deb安装方法(需要指定版本，还是按照提供的参考进行安装)。
- 登记服务器公钥

```
sudo apt-key adv --keyserver keys.gnupg.net --recv-key F6E65AC044F831AC80A06380C8B3A55A6F3EFCDE || sudo apt-key
 adv --keyserver hkp://keyserver.ubuntu.com:80 --recv-key
```

1. 添加将库添加到服务器库列表中

```
sudo add-apt-repository "deb http://realsense-hw-public.s3.amazonaws.com/Debian/apt-repo xenial main" -u
```

1. 安装库

```
sudo apt-get install librealsense2-dkms
sudo apt-get install librealsense2-utils
# Optionally
sudo apt-get install librealsense2-dev
sudo apt-get install librealsense2-dbg
```

1. 测试安装是否成功

重新连接D415（拔插），启动realsense-viewer(命令行输入)。

1. 查看驱动版本

```
modinfo uvcvideo | grep "version:"
```

会出现

```
version:        1.1.1
srcversion:     FC28E0DFD6253AEC0D4A815
```

- 可能遇到的问题

使用 `roslaunch ORB_SLAM2 rgbd.launch` 时，可能会报错

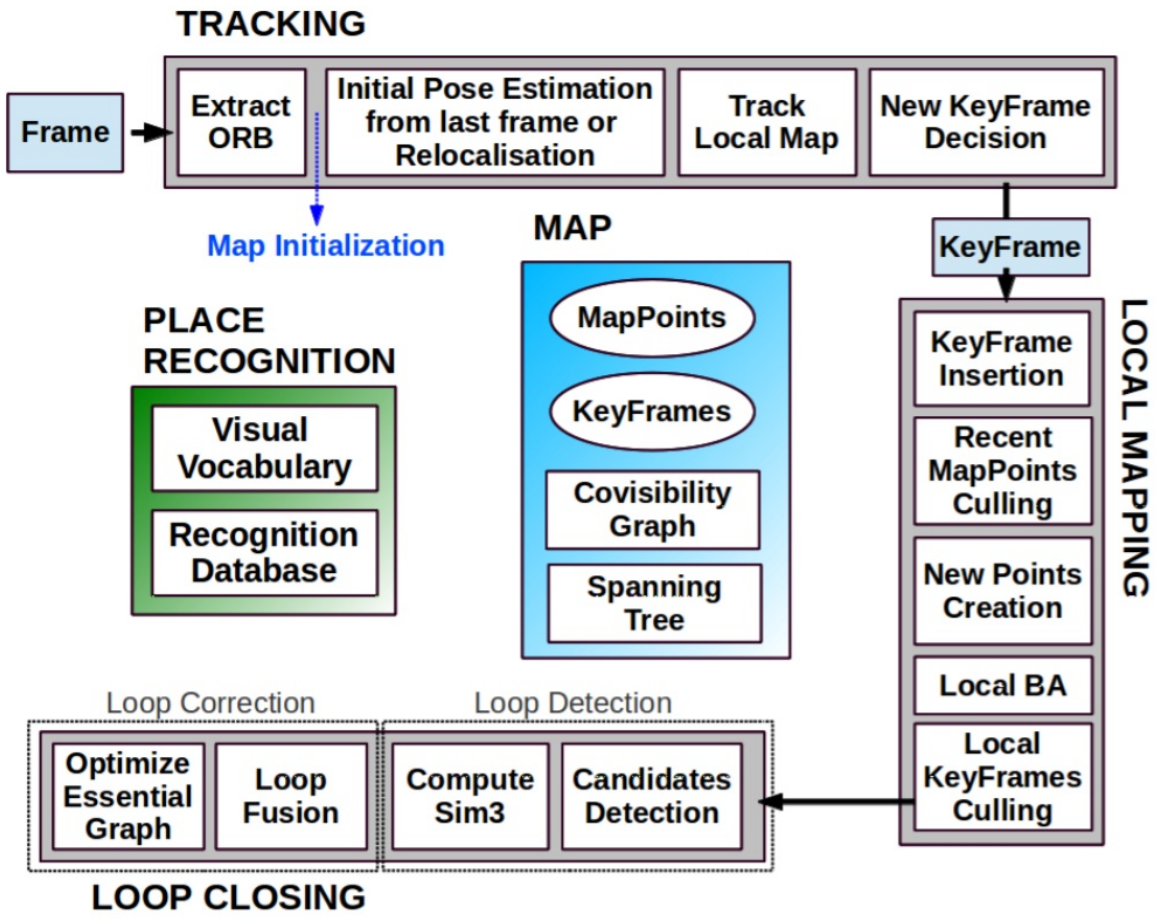
```
librealsense2.so.2.25: cannot open shared object file: No such file or directory
# 问题来源
warning: librealsense2.so.2.25, needed by /home/cwj/Repo/Pangolin/build/src/libpangolin.so, not found (try usin
g -rpath or -rpath-link)
# 解决方法
重新编译pangolin & ORB_SLAM2的ROS版本，注意将之前的文件删除。
```

4.5 ORB-SLAM & ORB-SLAM2 代码结构

4.5.1 Introduction

在第一节讲义中，解释了ORB特征，这只是万里长城的第一步，我们也看到了ORB-SLAM的基本框架，Tracking, Mapping & Loop detection，下面，将以代码的整体框架为核心，探究ORB-SLAM是如何实现这几部分功能的。

4.5.2 ORB-SLAM1流程



之前已经提到过，ORB是三个线程并行运算，包括*Tracking, Local Mapping, Loop Closing*，这个框图概括了整体流程。

- Tracking

首先是摄像头的图像输入，然后从输入图像中提取ORB特征，开始进行单目的初始化，单目的初始化一般是通过对极约束的。

$$ 基础矩阵 F= K^{-T} t^{\wedge}RK^T, 本质矩阵E=t^{\wedge}R

$$ 当特征点在平面内时，加入平面约束

$$ n^TP+d =0,
单应矩阵H=K(R-\cfrac{tn^T}{d})K^{-1}

$$ ORB-SLAM1会同时进行两种初始化，并将求得结果评价，判断使用那种方式，评价公式是

$$ S*M* = *sum_i(\rho_M(d*{cr}^2(x*c^i,x_r^i,M))+\rho_M(d*{rc}^2(x_c^i,x_r^i,M))) \ M\isin[F,H] \ \rho_M(d^2)= \begin{cases} \tau - d^2, d^2 < T_M \ 0, d^2 \eqslantgtr T_M \end{cases} \ R_H = \cfrac{S_H}{S_H+S_F}\ d_cr^2:相似变换误差 \ T_M:外点阈值 \ \begin{cases} R_H > 0.45, 本质矩阵 \ others, 单应矩阵 \end{cases}

$$

当选择基础矩阵初始化时，采用八点法进行位姿求解，采用本质矩阵时，从同样的样本中取四对点初始化。

初始化后开始Tracking，有两种模式，之前的跟踪成功的话，可以直接用一个常量速度模型预测下一时刻位姿和特征点的位置，进行优化，如果失败的话，在上一帧附近扩大范围搜索；如果是重定位状态下，也就是丢失追踪后，需要通过关键帧的词袋查询，匹配认知库中相符的位置，对一致性估计。

通过两种位姿估计，可以得到相机的位姿和初始的一系列特征匹配，然后开始Tracking local map，将与当前关键帧有相同地图点的其他关键帧形成一个子集K1, 将共视图中与K1相邻的关键帧形成一个子集K2，从K1中取出大部分与当前关键帧共有地图点的部分形成一个子集Kref，通过K1和K2地图点在当前帧的投影，舍弃部分点（选取方法见[1]），使用当前的地图点，优化相机位姿——motion-only BA。

追踪局部地图后，新的帧不断进入，需要判断哪一帧是关键帧，也就是将普通帧插入为关键帧，必要条件是：

1. 从上一次全局重定位后，已经有至少20帧进入；
2. 局部地图空闲或者从上一次插入关键帧后已经有新的至少20帧进入；
3. 追踪至少50个地图点；
4. 追踪的点和Kref的重复率不能高于90%。

插入新的一个关键帧后，就属于Local Mapping的线程了。

- Local Mapping

有了以上给出的新关键帧，将新的关键帧更新到共视图中，更新节点和边信息，更新生成树(spanning tree),新地图点需要经过筛选，决定是不是能保留在地图上，条件是：

1. 地图点必须有多于25%在预测可观测区中能被追踪；
2. 至少在三个关键帧中出现。

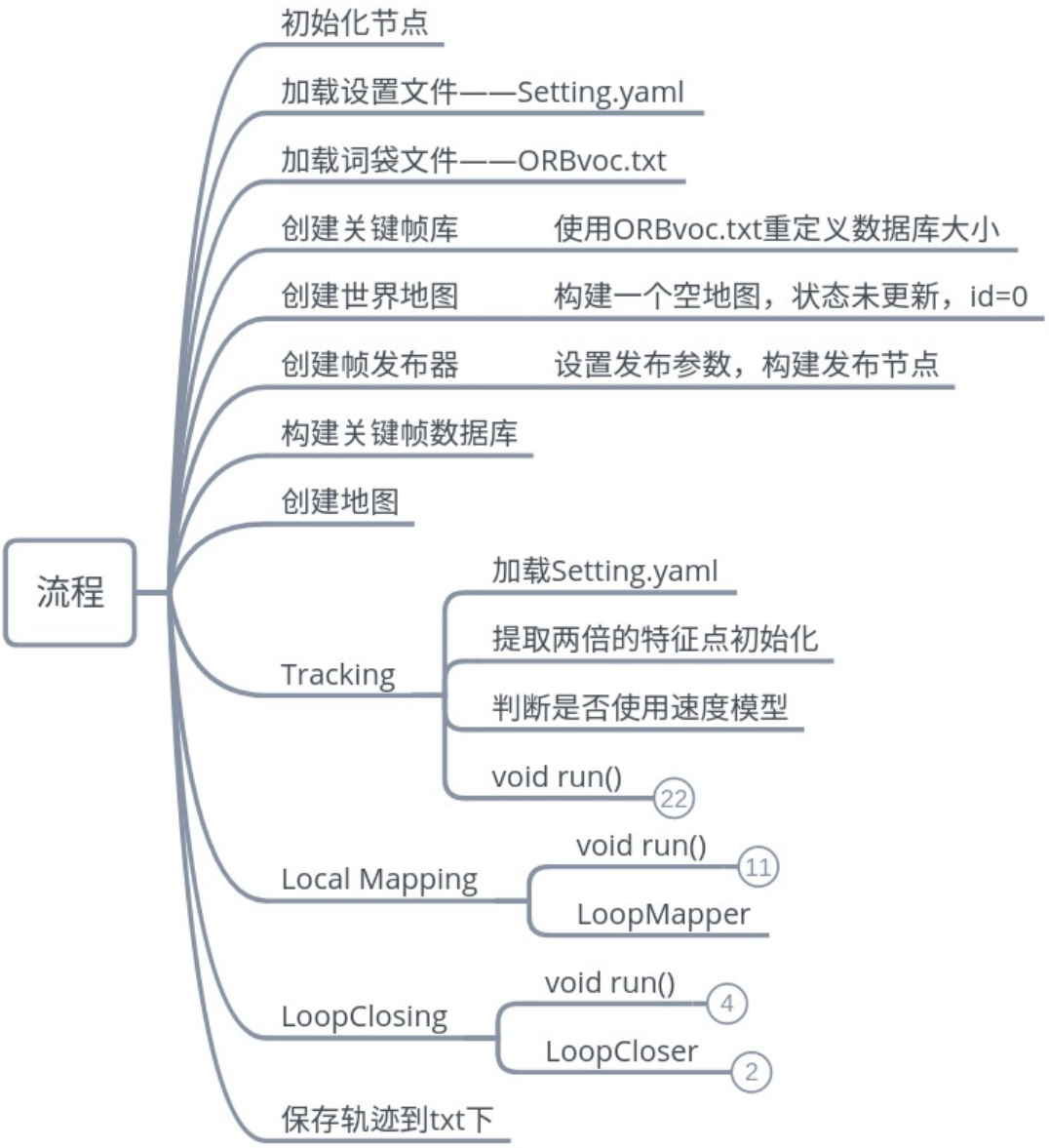
新生成地图点的方式是，从ORB特征中匹配三角化，验证它在其他关键帧中的投影，判断一致性。有了地图点和关键帧后，需要进行非线性优化——局部BA，保持关键帧不动，优化全部局部地图点。为了保持结构的精简，BA问题的复杂度在一个较低的程度，需要对冗余的关键帧进行筛除，舍弃与当前处理帧相关连的共视图中其他关键帧，因为有90%以上的点，与当前关键帧重复。

- Loop Closing

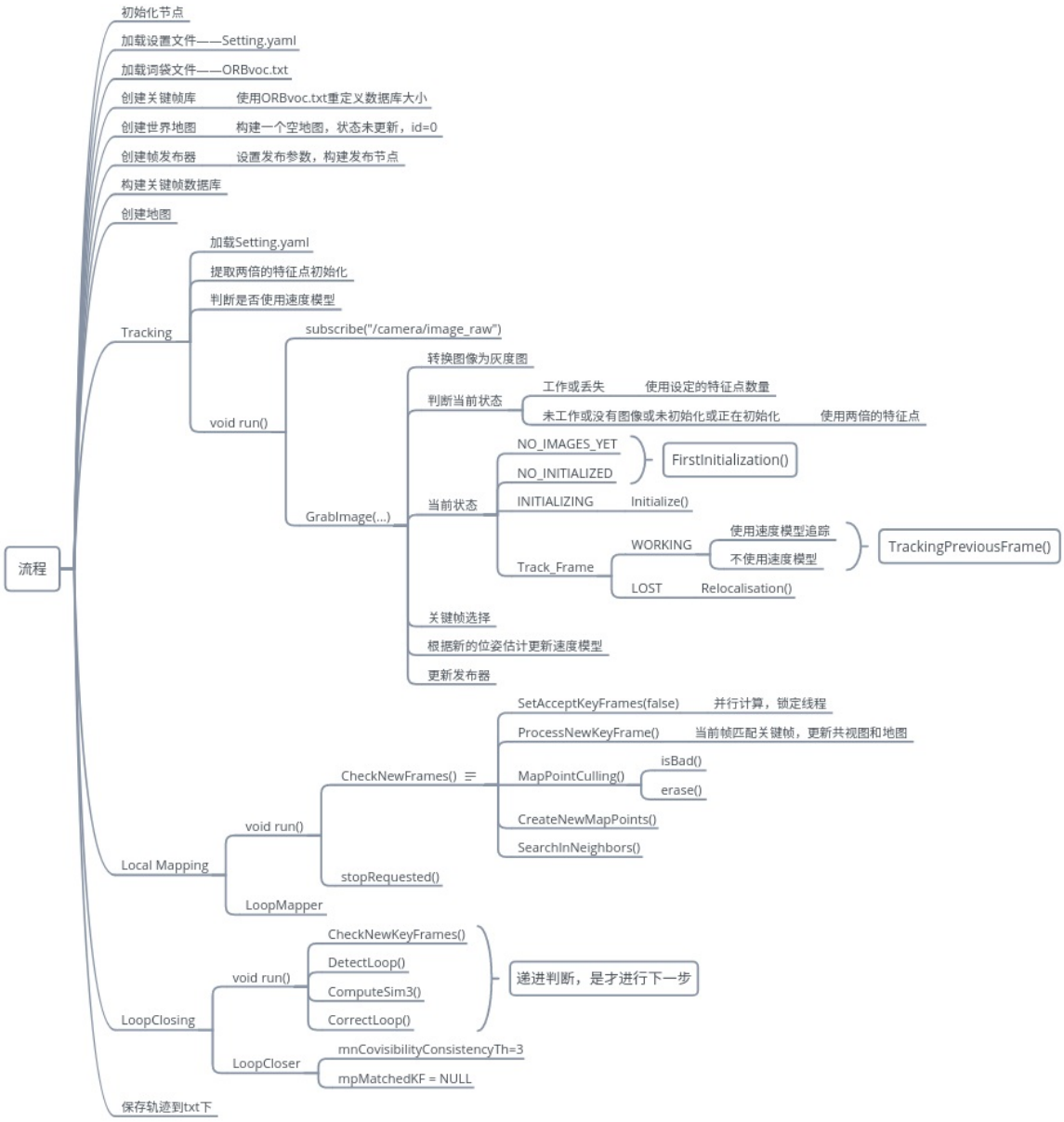
完成局部建图后，需要进行回环检测，方法是计算当前关键帧的词向量，与所有的共视图相邻关键帧词向量的相似变换，可以得到几个闭环的候选者，检测候选者共视图相邻的三个关键帧，判断是否可以作为闭环候选，对于当前的关键帧Ki可能会有多个相似的场景,需要进行多次的检测回环，计算Ki和闭环关键帧Kl间的相似变换，执行RANSAC迭代，当发现某个相似变换Sil有足够的内点，将Kl作为回环接受。

形成回环后，需要对重复的点进行合并，在共视图中插入新的边，完成回环。用Sil矫正当前的位姿，同时将矫正传递到Ki的附近关键帧，更新整个回环点和边。然后使用本质图优化，离散化整个回环的误差到每个节点和边，优化所有的地图点和关键帧。

4.5.3 Codes脉络



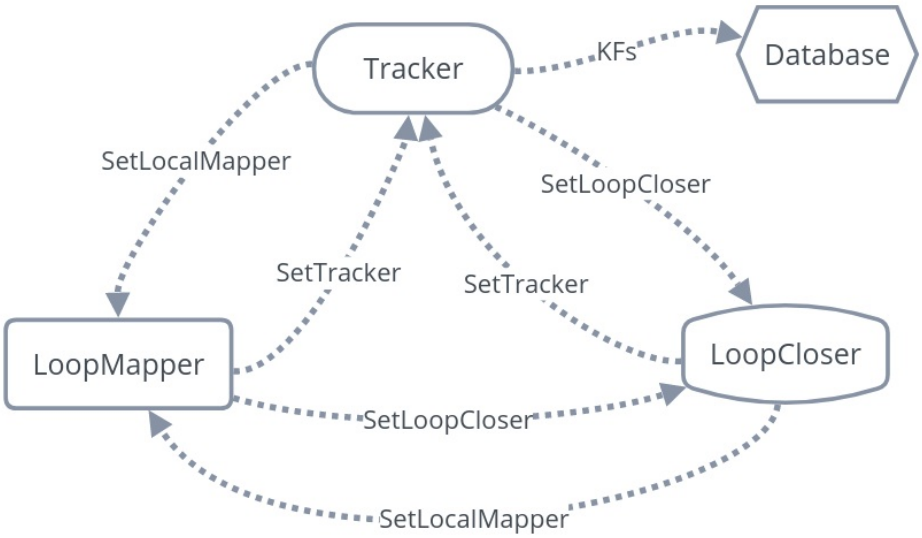
以上图片相当于将代码转化成了一个思维导图，可以从流程图的角度来看这幅图，即从上到下，一步一步执行，从这个图中也可以看出，ORB的代码逻辑十分友好，封装很强，尤其是使用三个线程，在代码中体现的也非常出色。



从开始的初始化节点，到加载词库，创建关键帧数据库等等，都可以对应到**src/**下的**.h 和.cpp**文件，这部分就不展开了，可以对应的去看src/。

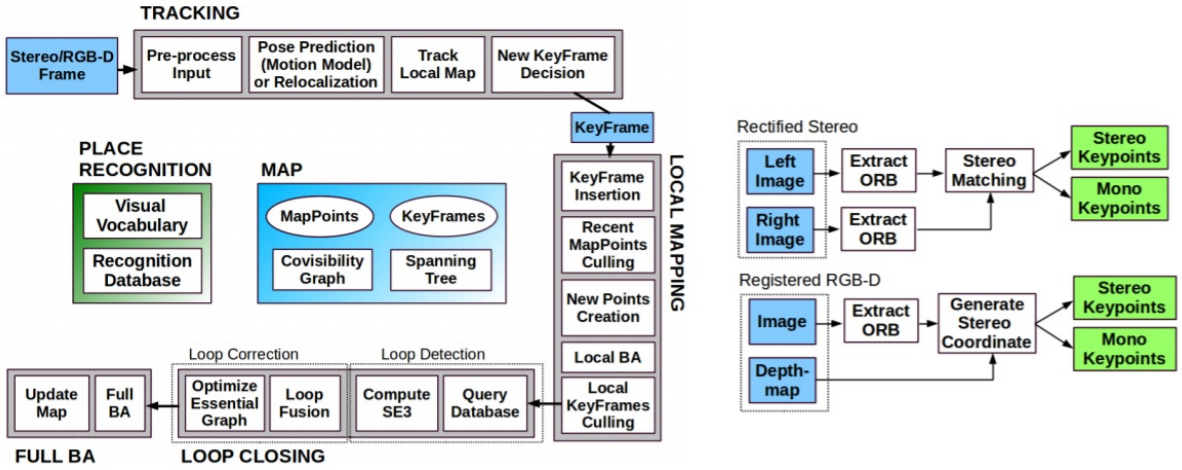
```
//Initialize the Tracking Thread and launch
    ORB_SLAM::Tracking Tracker(&Vocabulary, &FramePub, &MapPub, &World, strSettingsFile);
    boost::thread trackingThread(&ORB_SLAM::Tracking::Run,&Tracker);

    Tracker.SetKeyFrameDatabase(&Database);

    //Initialize the Local Mapping Thread and launch
    ORB_SLAM::LocalMapping LocalMapper(&World);
    boost::thread localMappingThread(&ORB_SLAM::LocalMapping::Run,&LocalMapper);

    //Initialize the Loop Closing Thread and launch
    ORB_SLAM::LoopClosing LoopCloser(&World, &Database, &Vocabulary);
    boost::thread loopClosingThread(&ORB_SLAM::LoopClosing::Run, &LoopCloser);
```

核心部分就是这三个并行线程，通过各自的run()函数，启动线程。可以根据图去找对应的函数，然后看看它们都做了什么工作。



这三个线程，通过指针相互传递数据，完成联系。

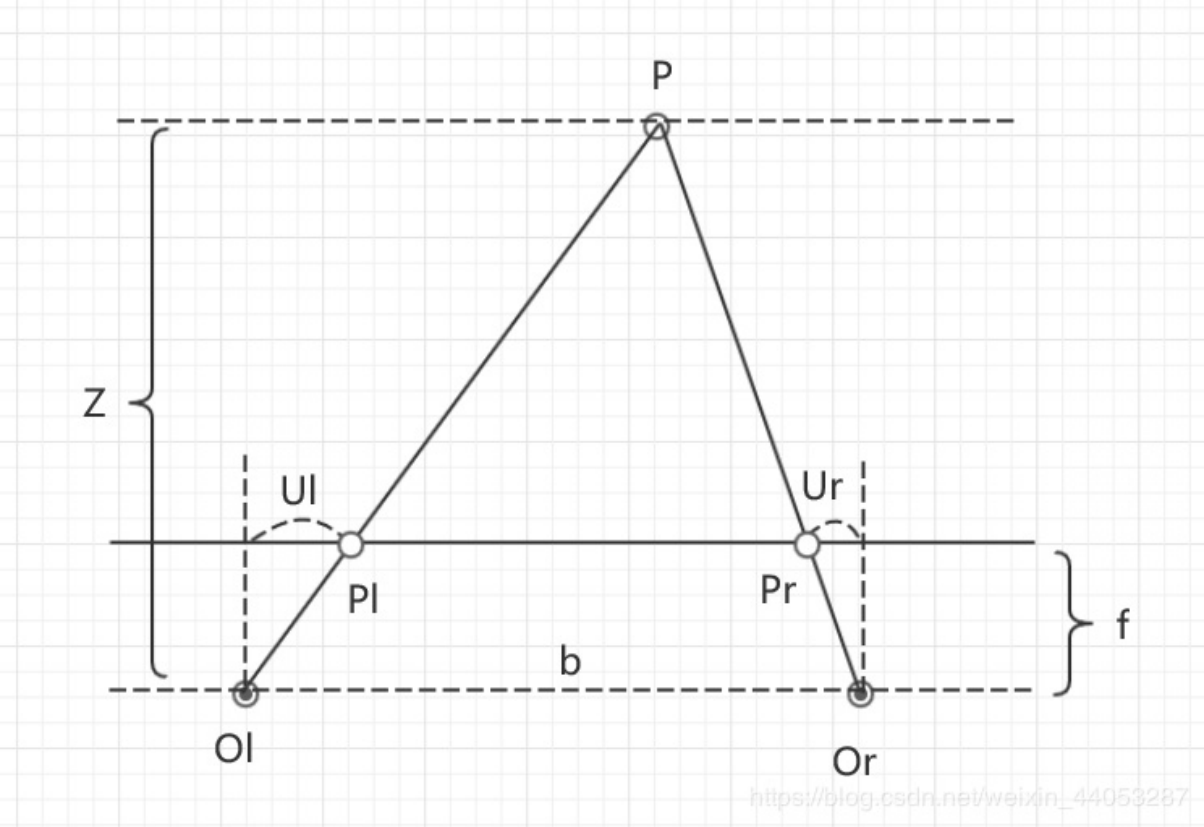
4.5.4 ORB2的不同之处



(a) System Threads and Modules. (b) Input pre-processing

左侧的流程，大同小异，可以从system.cc中看出来，主要的区别在于双目和RGB-D的支持，所以重点关注一下右侧的这幅处理立体点和RGB-D点的过程。

先回顾一下双目模型是如何获取深度的[1] 。



P点是世界坐标系下的一个地标点(landmark)，Ol是左目光心，Or是右目光心，Z是深度，f是焦距，Pl, Pr分别是左右相机成像平面形成的投影，Ul, Ur分别是左右的水平方向像素坐标，还有一个垂直方向的像素坐标Vl, b是基线也就是两个相机光心的距离。

根据相似三角形，可以得到

$$ \frac {z-f}{z} = \frac{b-u_l+u_r}{b} \ z=\frac{fb}{d},d=u_l-u_R

$$ d是左右图横坐标像素差，称为视差(*Dispartiy*)[2]。

如果是双目模型的话，可以得到左右两幅图像，从**left image**中提取特征，在**right image**中匹配，通过

$$ (U_l, V_l, U_r)

$$

进而得到的特征点的立体表示。对于RGB-D相机，提取RGB image中的ORB 特征，将深度图像中的深度d，通过双目模型的公式转化一个虚拟的右目图像：

$$ u_r = u_l - \frac {f_x b}{d}

$$ 这个变换是为了将RGB-D中depth的不确定性，转化为右目的像素不确定性，可以和双目模型进行相同的优化处理。

除了stereo point外，还有一个monocular point，这个点就是双目的左目或者RGB-D的RGB image，当双目模型或者RGB-D的匹配出现问题时，通过单目的三角化来还原位姿变化。

相应的在Frame.cc中构建了stereo & RGBD的Frame::Frame()类成员函数。

4.5.4 总结

ORB-SLAM突出的特点是总有plan B作为救火队员，从初始化的两种方式，到双目和RGB-D的单目补救手段，考虑到了现实场景中尽可能多的情况，所以鲁棒性才会如此优秀。代码的书写也十分条例清晰，可以逐个模块寻找对应的实现方法，无怪乎能成为脱颖而出了。

参考

[1]. 图片来源于百度 ↩

[2]. 视觉SLAM十四讲从理论到实践，p91 ↩

VINS-Mono算法

本章简介

不同于前面几章介绍的VSLAM算法，本章介绍的VINS-Mono算法是以视觉传感器与惯性测量单元的数据共同作为系统的信息源，通常我们称为VIO算法。

本章首先对VINS-Mono算法理论部分加以简要介绍，介绍了VIO系统的定义、松耦合与紧耦合的区别以及对VINS-Mono算法框架进行了介绍。其次，我们介绍了如何安装编译VINS-Mono，并介绍如何利用公开数据以及实际的摄像头测试此算法，最后，我们给出了在整个实践过程中可能遇到的问题及解决办法以供大家参考。

5.1 VINS-Mono算法理论介绍

5.1.1 预备知识

视觉与IMU

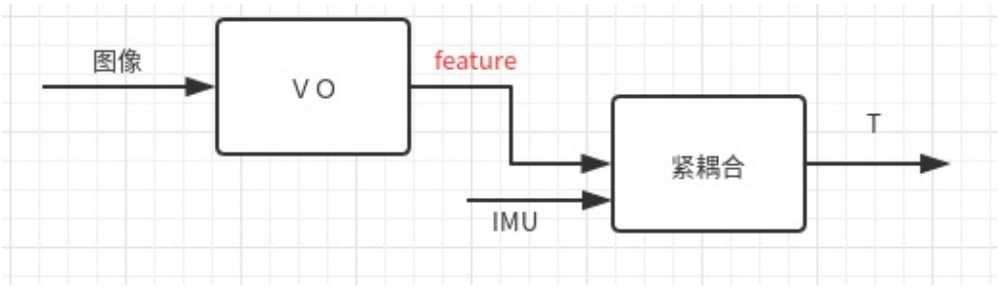
多传感器信息融合是当下的一个流行趋势，在机器人主体上一般也携带了多种传感器来协同完成任务。相机能获取图像数据，比如图像的特征点或者像素，来估计相机的运动，而惯性测量单元(IMU)能够测量传感器本体的角速度和加速度，它们通常被认为具有明显的互补性，在此，我们列出相机和IMU的优势和劣势对比表格如下：

	优势	劣势
相机	不产生漂移 直接测量旋转和平移	单目相机无法解决尺度问题 在纹理少的场景无法工作 快速运动会导致图像模糊跟踪易丢失 动态场景会产生错误匹配
IMU	快速响应 不受成像质量的影响 可估计绝对尺度	易受温度、零偏、振动的影响数据产生漂移 低精度IMU积分位姿发散 高精度IMU价格昂贵

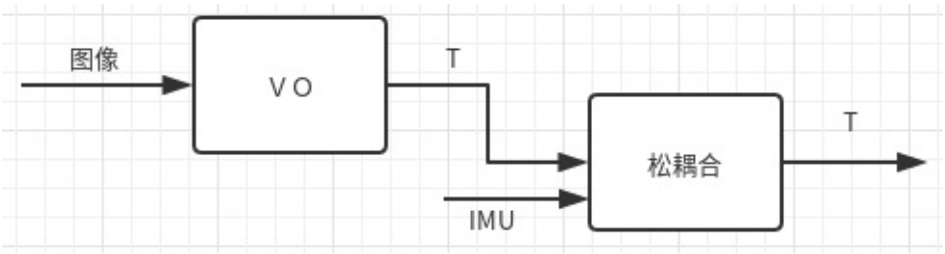
对于短时间快速的运动，相机采集到图像会产生模糊，无法准确的进行图像匹配，而IMU能够提供较好的估计。相比于IMU，相机数据基本不会有漂移，相机数据可以有效的估计并修正IMU在慢速运动或静止情况下读数的移。

松耦合与紧耦合

一般来说，根据后端的优化方式，我们将SLAM系统分为基于滤波与基于优化两种类型。在视觉与其它传感器融合的SLAM方案中，有将视觉约束加入到联合优化的结构我们称之为紧耦合，而把根据视觉约束求出的位姿再与其它传感器（这里指IMU）再进行融合求解出新的位姿的结构我们称之为松耦合。



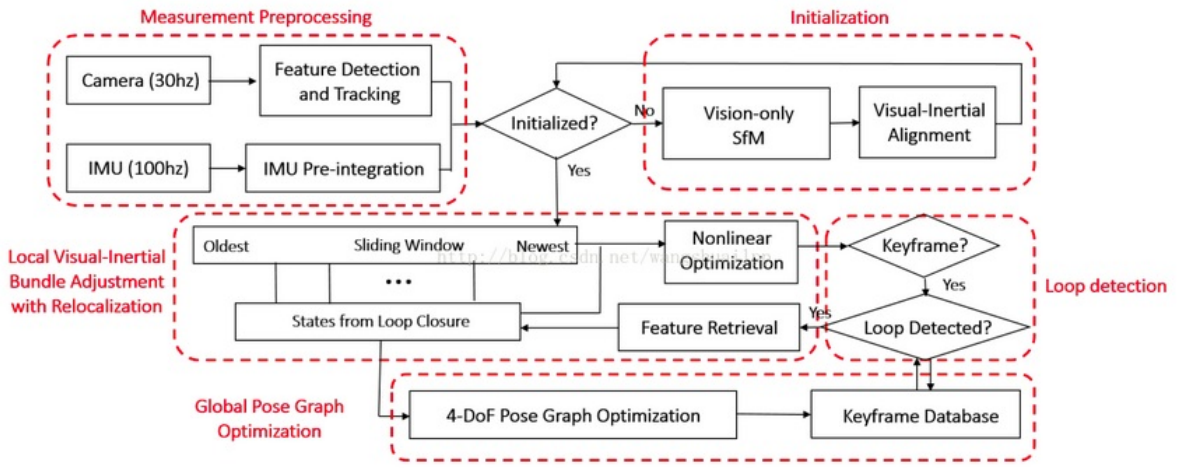
紧耦合



松耦合

VINS-Mono是香港科技大学沈劭劼团队在2018年开源的VIO方案。使用相机数据与惯性测量单元(IMU)数据进行融合，在后端滑动窗口内实现紧耦合，将滑动窗口内的所有优化变量共同进行优化。

5.1.2 算法总体框架



VINS-Mono的算法总体框图分为五个部分。

数据预处理：

图像：提取图像Harris焦点，利用金字塔光流跟踪相邻帧，再通过RANSAC去除外点，最后将跟踪到的特征点push到图像队列中，并通知后端进行处理。

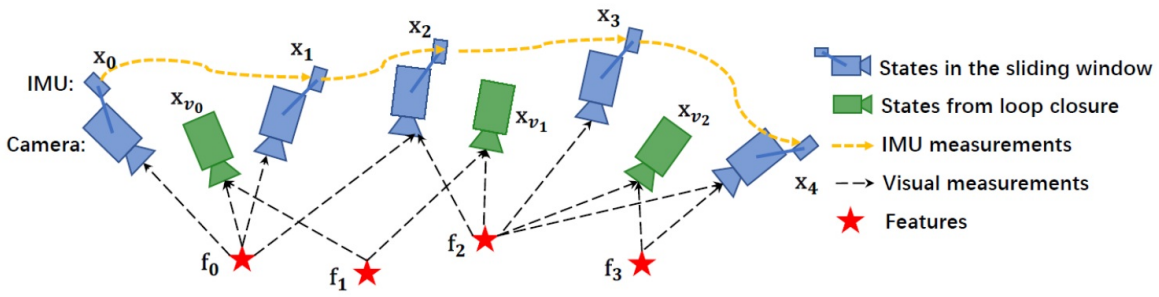
IMU：将 IMU 数据进行积分，得到当前时刻的位置、速度和旋转（PVQ），同时计算在后端优化中将用到的相邻帧的预积分增量，及预积分误差的 Jacobian 矩阵和协方差项。

初始化：

首先，利用 SFM 进行纯视觉估计滑窗内所有帧的位姿及 3D 点逆深度，最后与 IMU 预积分进行对齐求解初始化参数。

后端非线性优化：

将在滑动窗口内的视觉约束、IMU约束与闭环约束共同放在一个大的目标函数中进行线性优化，实现紧耦合，求解出滑窗内所有帧的 PVQ、bias 等。



滑动窗口优化示意图

闭环检测：

利用DBoW进行闭环检测，当检测成功后进行重定位。

全局位姿图优化：

之前做的非线性优化本质上只是在一个滑窗内求解出了相机的位姿，而且在回环检测部分，利用固定位姿的回环帧只是纠正了滑窗内的相机位姿，并没有修正其他位姿(或者说没有将回环发现的误差分配到整个相机的轨迹上)，缺少全局的一致性，所以要做一次全局位姿图优化。

5.1.3 总结与评价

VINS-Mono前端使用IMU预积分与视觉信息融合，后端通过采用滑动窗口与优化框架实现视觉与IMU的紧耦合，并且具备自动初始化，在线外参标定，重定位，闭环检测，以及全局位姿图优化功能，算是一套完整的SLAM系统。在当前开源的VIO算法中精度较好，能够在室内复杂环境和室外复杂且大尺度环境下稳定工作。

但是根据实际运行效果来看：在运行过程中，视觉跟踪失败即一小段没有特征点或者移动太快的时候IMU会扩散得很远；另外，尺度信息的获取是通过IMU预积分得到的，当IMU数据的漂移很大时，预积分获取的数据就会产生误差，影响整体算法的精度和鲁棒性。

5.2 安装编译运行VINS-Mono

5.2.1 现有环境介绍

在利用公开数据集对VINS-Mono算法进行测试之前我们的现有环境如下：

- ubuntu18.04
- ROS melodic
- opencv 3.4.5 （用于处理图像的开源计算机视觉库）
- Eigen3.3.4 （线性代数库）
- ceres-solver （非线性优化库）

5.2.2 安装ROS依赖包并编译

首先安装必要的依赖如下：

```
sudo apt-get install ros-melodic-cv-bridge ros-melodic-tf ros-melodic-message-filters ros-melodic-image-transpo
rt
```

其次建立工作空间，再克隆源代码到 `src` 文件夹中，回到工作空间目录，执行编译命令。

```
cd ~/workspace/src #已经建立过工作空间的此步骤跳过
git clone https://github.com/HKUST-Aerial-Robotics/VINS-Mono.git
cd ..
catkin_make
echo "source ~/workspace/devel/setup.bash" >> ~/.bashrc
```

5.2.3 在公开数据集上测试VINS-Mono

1. 下载公开数据集，下载地址为：

   https://projects.asl.ethz.ch/datasets/doku.php?id=kmavvisualinertialdatasets#downloads

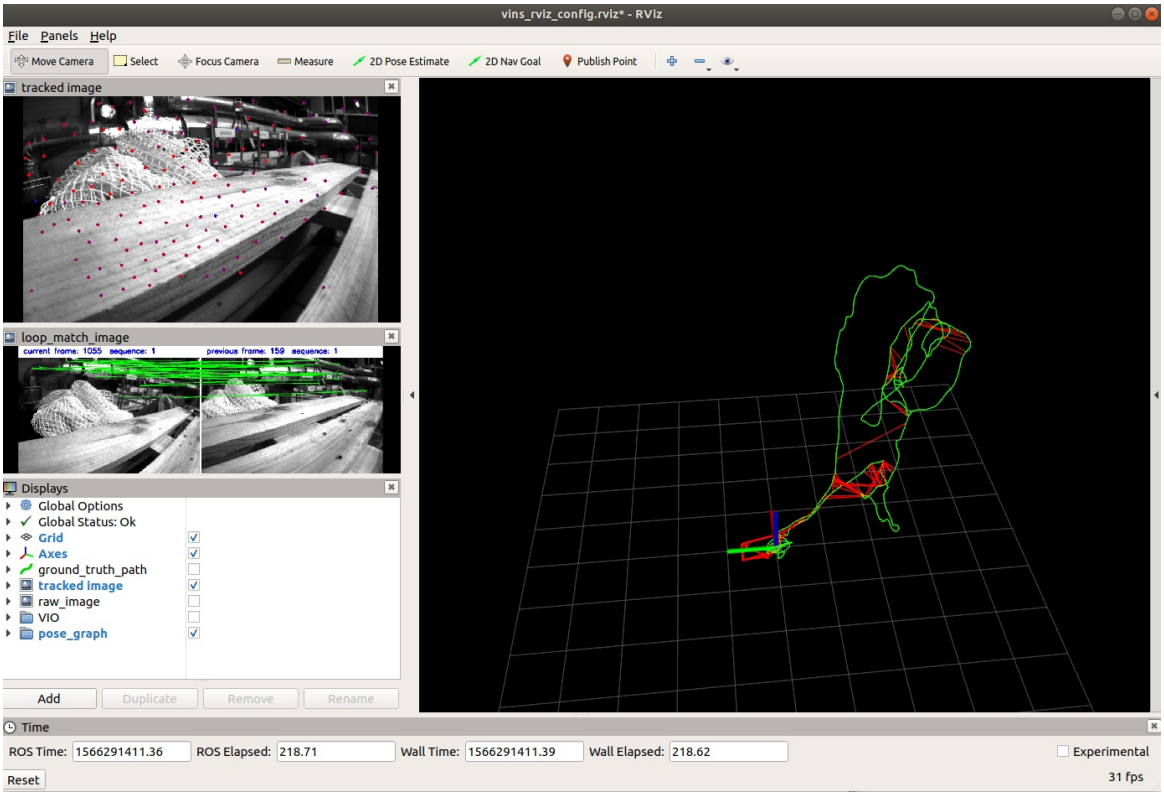
   建立Data文件夹，将下载好的数据集放在 `Data` 文件夹中。

   ```
   cd ~workspace/src/VINS-Mono
   mkdir Data
   ```

   ２．测试流程

   2.1.1 打开三个终端，分别启动 `vins_estimator` , `rviz` 和播放bag文件。以MH_01为例：

   ```
   roslaunch vins_estimator euroc.launch
   roslaunch vins_estimator vins_rviz.launch
   rosbag play ~/workspace/src/VINS-Mono/Data/MH_01_easy.bag
   ```

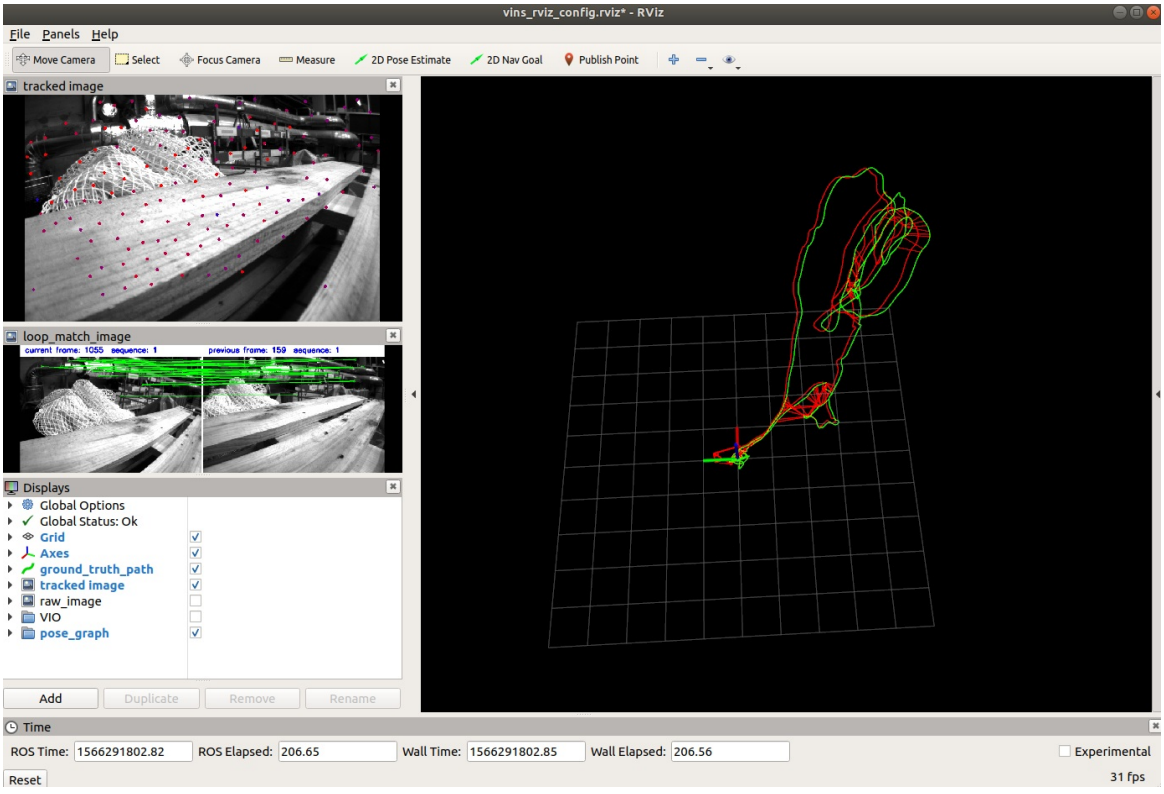


2.1.2 可视化ground truth

作者写了一个 benchmark publisher，以帮助可视化ground truth。

```
roslaunch benchmark_publisher publish.launch  sequence_name:=MH_01_easy
```

运行结果如下：



绿线是VINS结果，红线是ground truth。从运行过程来看，随着轨迹的增长，绿线与红线的差距越来越宽，但是当出现回环以后，算法能修正轨迹使其轨迹更接近于ground truth。

2.1.3 可以在没有相机和IMU之间外部参数的情况下运行EuRoc,VINS-Mono能进行在线校准。

将

```
roslaunch vins_estimator euroc.launch
```

替换成为：

```
roslaunch vins_estimator euroc_no_extrinsic_param.launch
```

该配置文件中没有外部参数。等待几秒钟进行初始校准。

2.2 地图合成

播放MH_01 bag后，可以继续播放MH_02 bag，MH_03 bag… 系统将根据回环合并它们。 地图合成的前提条件是要有重合的公共部分。

2.3地图保存 在 `VINS-Mono/config/euroc/euroc_config.yaml` 中设置 `pose_graph_save_path` 的路径。比如：

```
pose_graph_save_path: "/home/[用户名]/workspace/src/VINS-Mono/output/" # save and load path
```

播放MH_01 bag后，在vins_estimator终端输入s，然后按回车键，将保存当前位姿图。 注意：修改路径后，必须重新执行2.1.1中的操作，然后才能保存位姿图。

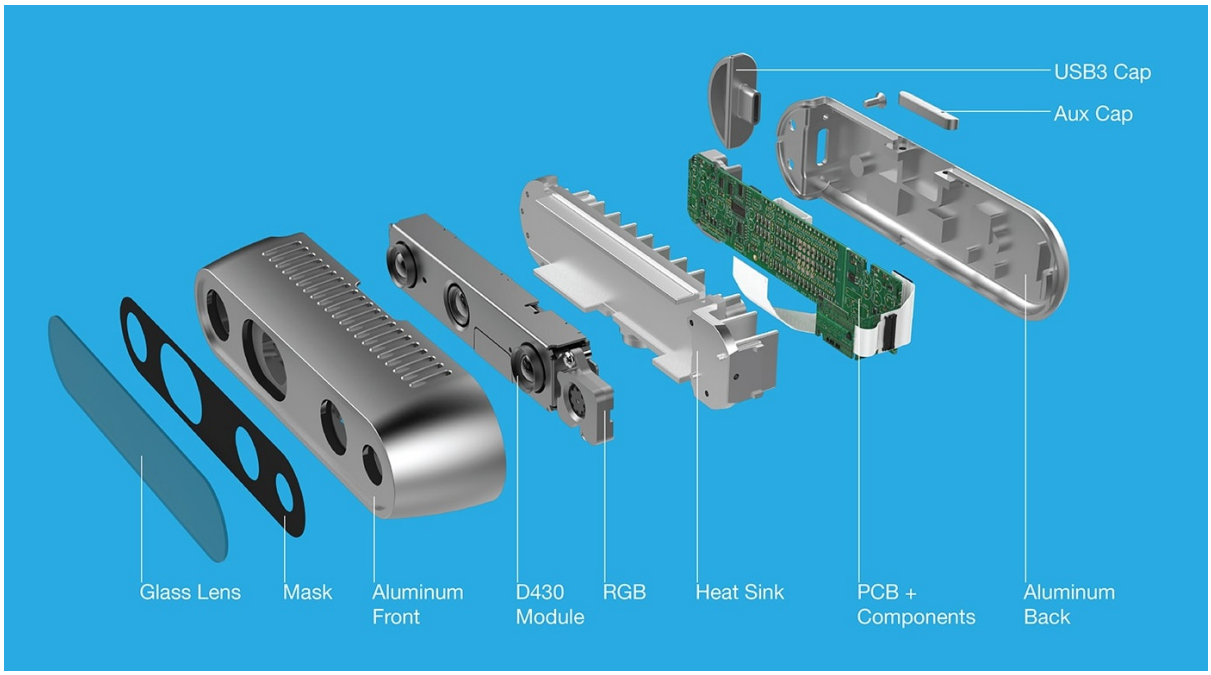
2.４　地图加载 在执行2.1.1之前将 `VINS-Mono/config/euroc/euroc_config.yaml` 中的load_previous_pose_graph设置为1。系统将从pose_graph_save_path加载先前的位姿图。然后可以播放MH_02 bag，新序列将与先前的位姿图对齐。

5.3 D435i运行VINS-Mono

5.3.1 D435i 介绍

D435i 是Intel Realsense 深度相机D435的升级版，它在D435的基础上额外搭载了IMU(惯性测量单元),它的外观和D435一样，如下图所示：

从左至右，第一个和第三个是红外传感器（IR Stereo Camera)，第二个是红外激光发射器(IR Projector)，第四个是彩色相机(色彩传感器)。下图是D435i的结构分解图：



从左至右分别是玻璃镜头、掩膜、铝制前盖、D430模块加RGB相机、散热器、PCB加组件、铝制后盖。

另外，D435i的2000万像素RGB摄像头和3D传感器可以以30帧/秒的速度提供分辨率高达1280 × 720的图像，或者以90帧/秒的速度提供848 × 480低分辨率图像。该摄像头具有全局快门，可以处理快速移动物体，室内室外皆可操作。深度距离在0.1 m~10 m之间，视场角度为85 × 58度。深度误差小于2%的特性。

内置的IMU(博世BMI055)惯性测量单元，结合视觉数据可实现6DoF追踪功能。其中，IMU将各种线性加速度计和陀螺仪数据结合，可检测X，Y，Z三轴的旋转和平移，以及俯仰、横摇等动作。对于高级别的扫描，IMU还提供额外数据集，用于密集重建，并提供参考，以防相机追踪丢失。D435i可以帮助开发者构建更先进深度感知和追踪性能的解决方案，并用于无人机，机器人和AR/VR领域。

5.3.2 安装Realsense SDK2.0

librealsense SDK相当于相机的驱动,D435i是librealsense SDK 2.0,我们采用命令行安装的方式来进行安装，步骤如下：

- 添加Intel 服务器到仓库列表

```
echo 'deb http://realsense-hw-public.s3.amazonaws.com/Debian/apt-repo xenial main' | sudo tee /etc/apt/sources.
list.d/realsense-public.list
```

- 注册公钥

```
sudo apt-key adv --keyserver keys.gnupg.net --recv-key 6F3EFCDE
```

- 更新仓库

```
sudo apt-get update
```

- 安装libraries

```
sudo apt-get install librealsense2-dkms
sudo apt-get install librealsense2-utils
```

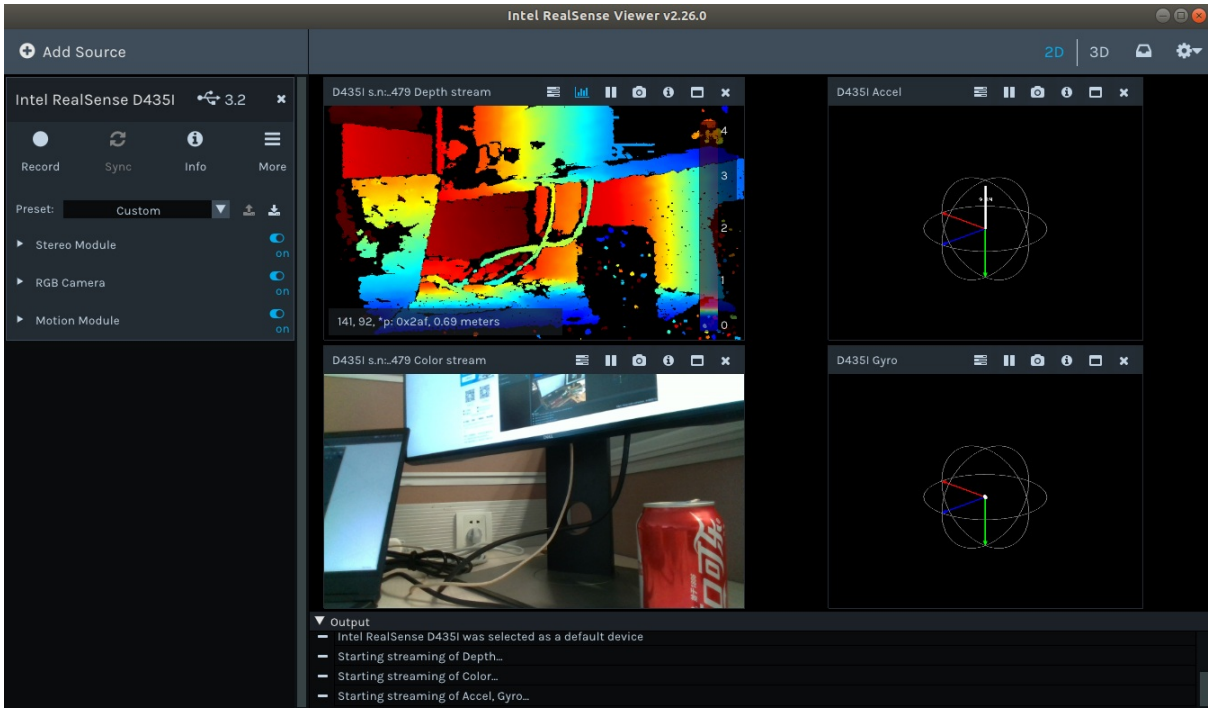
- 安装一些额外的package

```
sudo apt-get install librealsense2-dev
sudo apt-get install librealsense2-dbg
```

- 连接相机，输入以下命令

```
realsense-viewer
```

出现以下画面：



出现以上画面则表面安装成功。

5.3.4 安装ROS Wrapper

ROS Wrapper指相机对应的ROS版本的开发包，安装步骤如下：

- 建立工作空间(已经执行过的不需要再执行此步骤)

```
mkdir -p ~/workspace/src
cd ~/workspace/src/
catkin_init_workspace
cd ..
catkin_make
echo "source ~/workspace/devel/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

- 下载源码包

```
cd ~/workspace/src
git clone https://github.com/intel-ros/realsense.git
```

- 在ROS下编译

```
cd ..
catkin_make
```

- 启动相机节点

```
roslaunch realsense2_camera rs_rgbd.launch
```

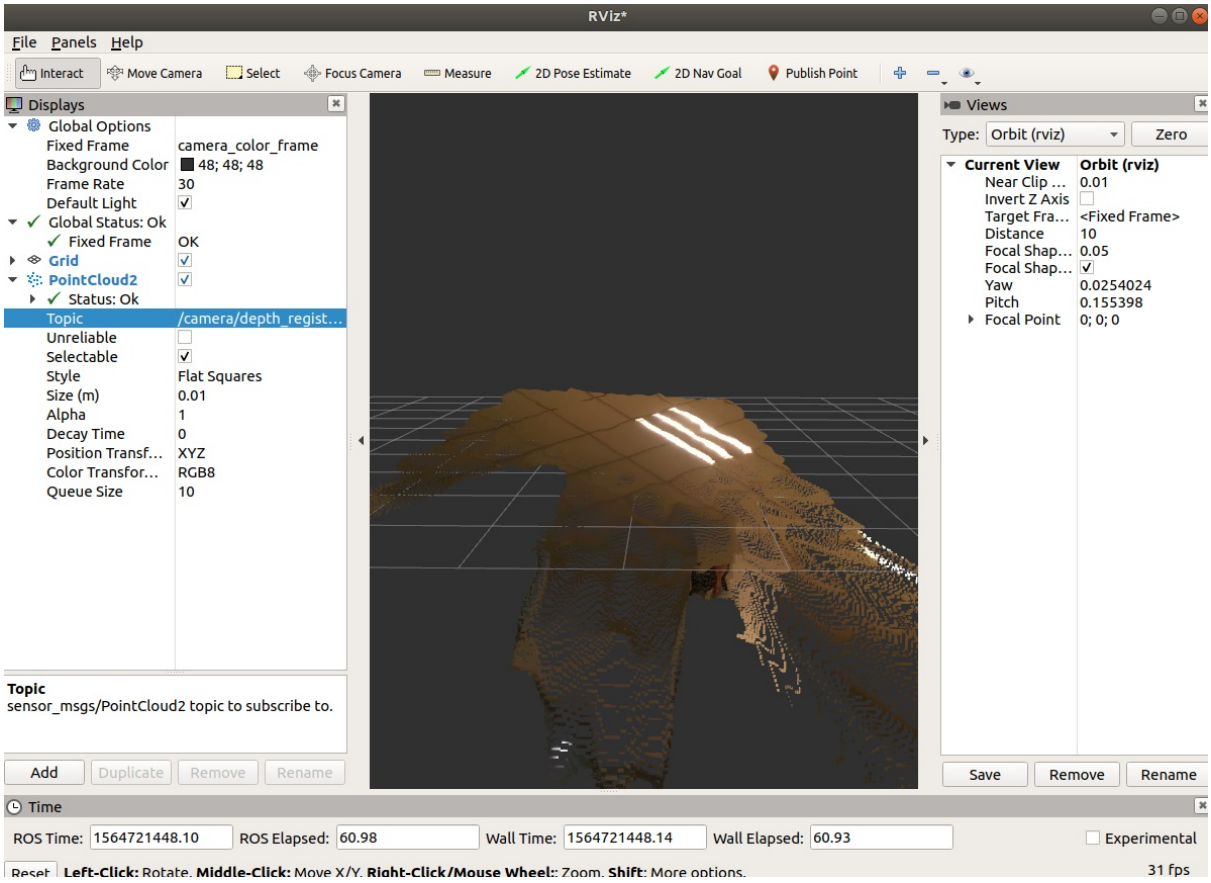
- 查看此时的topic,可以发现acc和gyr的信息是分别发布在两个topic上的。

```
rostopic list
```

```
/home/pxx/workspace/src/realsense/real...   pxx@pxx-GK5S: ~
xx@pxx-GK5S:~$ rostopic list
camera/accel/imu_info
camera/accel/sample
camera/aligned_depth_to_color/camera_info
camera/aligned_depth_to_color/image_raw
camera/aligned_depth_to_infra1/camera_info
camera/aligned_depth_to_infra1/image_raw
camera/color/camera_info
camera/color/image_raw
camera/color/image_rect_color
camera/color_rectify_color/parameter_descriptions
camera/color_rectify_color/parameter_updates
camera/depth/camera_info
camera/depth/image_rect_raw
camera/depth_registered/points
camera/extrinsics/depth_to_color
camera/extrinsics/depth_to_infra1
camera/extrinsics/depth_to_infra2
camera/gyro/imu_info
camera/gyro/sample
camera/infra1/camera_info
camera/infra1/image_rect_raw
camera/infra2/camera_info
camera/infra2/image_rect_raw
```

- 通过rviz查看图像

```
rosrun rviz rviz
```



执行上述命令之后，添加PointCloud2并订阅相关话题出现图像。

另外，若不对相机进行标定，可以先从Realsense ROS Wrapper发布的topic中获得相机的内参，执行一下命令查看相机参数

```
rostopic echo /camera/color/camera_info
rostopic echo /camera/aligned_depth_to_color/camera_info
```

5.3.5 在D435i上运行VINS-Mono

1. 修改realsense包里的rs_camera.launch文件

   1.1 打开rs_camera.launch文件

   ```
   cd ~/workspace/src/realsense/realsense2_camera/launch/
   gedit ./rs_camera.launch
   ```

   1.2 修改 `unite_imu_method` 如下，目的是为了让IMU的角速度和加速度合起来作为一个topic输出

   ```
    <arg name="unite_imu_method"      default="copy"/>
   ```

   1.3 修改 `enable_sync` 参数为true，目的的开启相机和IMU的同步

   ```
    <arg name="enable_sync"           default="true"/>
   ```

2. 修改VINS-Mono包里的realsense_color_config.yaml文件

   2.1 修改订阅的topic

   ```
   imu_topic: "/camera/imu"
   image_topic: "/camera/color/image_raw"
   ```

   2.2 获取相机内参，通过读取camera_info得到或者自己标定

   通过camera_info得到方式如下：

   打开一个终端

   ```
   roslaunch realsense2_camera rs_camera.launch
   ```

   再打开一个终端

   ```
   rostopic echo /camera/color/camera_info
   ```

   获取相机参数之后，修改 `realsense_color_config.yaml` 中的相应位置。

   2.3 IMU到相机的变化矩阵

   IMU和camera之间的外参矩阵建议使用Kalibr工具进行离线标定，（也可以将realsense_color_config.yaml中的estimate_extrinsic改成1或者2让估计器自己优化。）

   ```
   estimate_extrinsic: 2   # 0  Have an accurate extrinsic parameters. We will trust the following imu^R_cam,
   imu^T_cam, don't change it.
                           # 1  Have an initial guess about extrinsic parameters. We will optimize around your
    initial guess.
                           # 2  Don't know anything about extrinsic parameters. You don't need to give R,T. We
   ```

```
 will try to calibrate it. Do some rotation movement at beginning.
#If you choose 0 or 1, you should write down the following matrix.
```

2.4 IMU参数修改，这里使用了注释给定的参数

```
#imu parameters       The more accurate parameters you provide, the better performance
acc_n: 0.2          # accelerometer measurement noise standard deviation. #0.2
gyr_n: 0.05         # gyroscope measurement noise standard deviation.     #0.05
acc_w: 0.02         # accelerometer bias random work noise standard deviation.  #0.02
gyr_w: 4.0e-5       # gyroscope bias random work noise standard deviation.     #4.0e-5
g_norm: 9.80       # gravity magnitude
```

2.5 D435i已经做好了硬件同步不需要在线估计同步时差

```
#unsynchronization parameters
estimate_td: 0                      # online estimate time offset between camera and imu
td: 0.000                           # initial value of time offset. unit: s. readed image clock + td = real
 image clock (IMU clock)
```

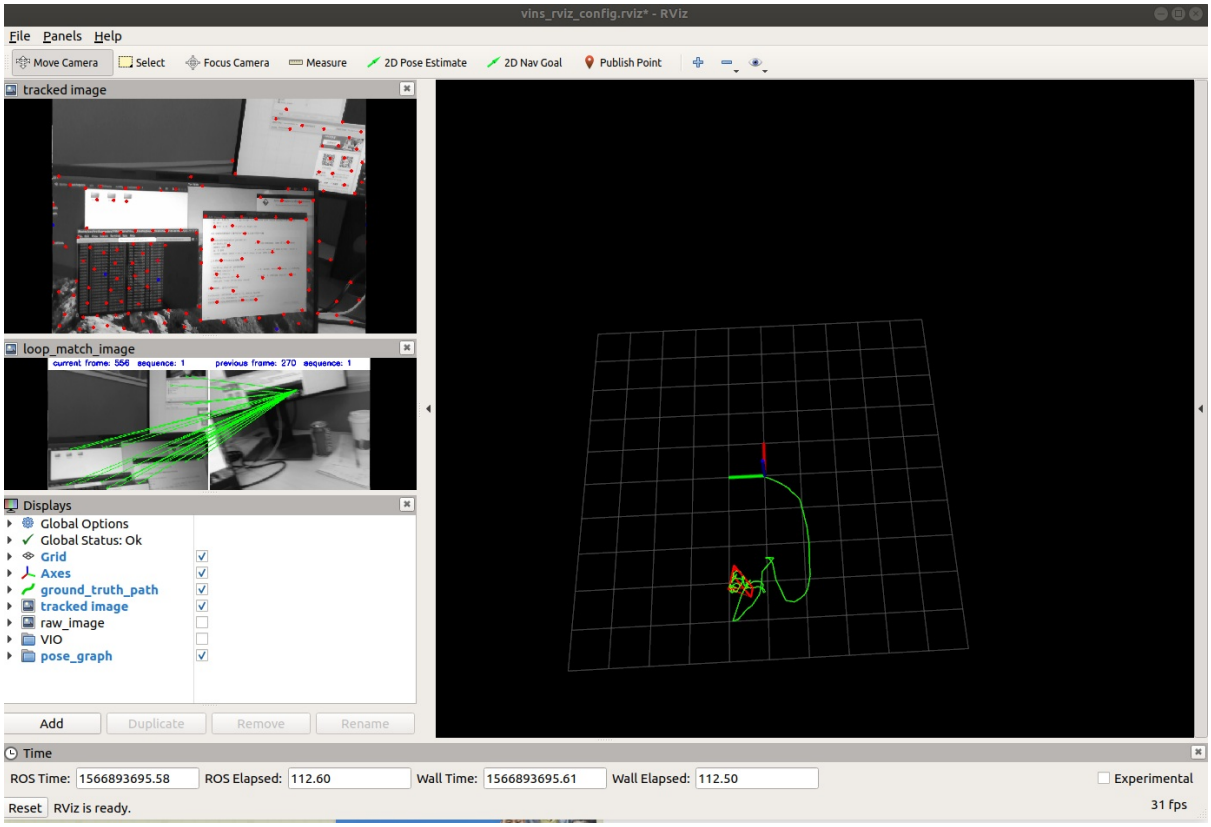
2.6 相机曝光方式改成全局曝光

```
#rolling shutter parameters
rolling_shutter: 0                      # 0: global shutter camera, 1: rolling shutter camera
rolling_shutter_tr: 0                   # unit: s. rolling shutter read out time per frame (from data sheet)
.
```

3. 连接相机，运行VINS-Mono

```
roslaunch realsense2_camera rs_camera.launch
roslaunch vins_estimator realsense_color.launch
roslaunch vins_estimator vins_rviz.launch
```

运行画面如下：



从实际运行运行效果来看，初始化需要一定时间，运行过程中，视觉跟踪失败即一小段没有特征点或者移动太快的时候IMU会飘的很离谱。

5.4 VINS-Mono安装编译运行常见错误及解决办法

Q1： 安装opencv的依赖时找不到 `libjasper-dev`

ANS: 下载安装即可，具体操作如下，在终端执行：

```
sudo add-apt-repository "deb http://security.ubuntu.com/ubuntu xenial-security main"
sudo apt update
sudo apt install libjasper1 libjasper-dev
```

其中libjasper1是libjasper-dev的依赖包。

Q2: 安装IntelRealse D435i的ROS Wrapper时缺少 `ddynamic_reconfigure`

ANS: 从https://github.com/pal-robotics/ddynamic_reconfigure/tree/kinetic-devel 下载安装包,

解压在 `~/workspace/src` 文件夹下，再catkin_make即可。

Q3: 安装IntelRealse D435i的ROS Wrapper时缺少 `cv_bridge`

ANS: 终端执行

```
sudo apt-get install ros-kinetic-cv-bridge
```

即可解决。

Q4: 在启动相机节点时出现找不到 `rgbd-launch`

ANS: 终端执行

```
sudo apt-get install ros-kinetic-rgbd-launch
```

Q4: 找不到 `tf` 包

ANS: 终端执行

```
sudo apt-get install ros-indigo-tf2-*
```

即可解决。